Lerneinheit 3 · NumPy und „Tensor Thinking“

M2 · Data-Science-Werkzeugkasten

Version 1.0

Lernziele

Nach dieser Lerneinheit können Sie …

- ☐ ein Array nach Form, Datentyp, Speicherbedarf und Speicherlayout beschreiben,
- ☐ die semantische Suche aus der letzten Lerneinheit als Array-Rechnung in NumPy berechnen,
- ☐ die Ergebnisform einer Broadcasting-Operation vorhersagen und eine stille Broadcasting-Falle erkennen,
- ☐ den Operator `row_l2_normalize` nach seinem Vertrag implementieren und an den Randfällen prüfen,
- ☐ an einem Ausdruck entscheiden, ob er eine Sicht oder eine Kopie liefert,
- ☐ technisch begründen, warum die Array-Fassung schneller ist als die Schleife,
- ☐ die wichtigsten Python-Bibliotheken für Daten und KI nach ihrem Zweck einordnen.

In der letzten Lerneinheit haben Sie die semantische Suche in reinem Python geschrieben, mit Schleifen von Hand, und am Ende gesehen, dass kompakter Speicher allein die Rechnung nicht schneller macht. In dieser Lerneinheit kommt der zweite Teil dazu: Die Schleife läuft nicht mehr im Interpreter, sondern als fertige, in Maschinencode kompilierte Operation auf einem ganzen Zahlenblock. Das ist NumPy. Sie schreiben dieselbe Suche noch zweimal: zuerst mit den Bordmitteln von Python aus Grundkurs Teil 2 (`sum`, `zip`, Comprehensions), dann als Rechnung auf NumPy-Arrays. Dabei entsteht `row_l2_normalize`, der Operator, der uns durch das ganze Semester begleitet: Er normalisiert jede Zeile einer Matrix, und Sie werden ihn später in PyTorch, CUDA und Mojo noch einmal schreiben. Auf der Landkarte rücken wir damit eine Station nach unten, von Python zu NumPy.

Ein Element zieht sich durch das ganze Notebook: die **Formenzeile**. Das ist kein Begriff von NumPy, sondern eine Konvention dieses Kurses: Nach jeder wichtigen Zelle steht als Kommentar, welche Formen die Arrays haben, zum Beispiel `E (400, 384) -> norms (400, 1)`. Rechnungen auf Arrays plant man am besten über ihre Formen, bevor man den Code schreibt. Diese Denkweise hilft uns später in PyTorch fast bei jeder Zeile.

Vorausgesetzt werden Teil 1 und Teil 2 des Python-Grundkurses. Aus Teil 2 brauchen Sie in dieser Lerneinheit List Comprehensions, Slicing und das Auspacken von Tupeln, alle drei frischt das Kapitel 3.1 auf.

Diese Lerneinheit ist dicht: Sie führt in einem Termin die Denkweise ein, mit der im ganzen Semester gerechnet wird, in NumPy genauso wie später in PyTorch und auf der GPU. Der Aufwand lohnt sich, weil alles Folgende darauf aufbaut.

Zwei Arten von Boxen helfen beim Einordnen. Boxen mit dem Titel „Vertiefung" beantworten Fragen für alle, die es genau wissen wollen; im Termin überspringen wir sie, und ihr Inhalt ist nicht prüfungsrelevant. Boxen mit dem Titel „Genau verstehen" sind das Gegenteil: Kernstoff, bei dem es auf die Einzelheiten ankommt. Das Kapitel „Zum Weiterüben" gehört nicht zum Termin, übt aber genau die Werkzeuge dieser Lerneinheit.

Inhalt

- 3.1 Aufwärmen
- 3.2 Drei Fassungen einer Suche
- 3.3 Das Array: ein Block mit Beschreibung
  - 3.3.1 Wie die Zahlen im Speicher liegen
  - 3.3.2 Vom Einzelwert zum Tensor
  - 3.3.3 Arrays anlegen, Zugriff und Slicing
  - 3.3.4 Achsen und Reduktionen
  - 3.3.5 Formen ändern: Transponieren und Umformen
  - 3.3.6 Vektorisierung und Prüfzellen
- 3.4 Der Python-Data-Science-Werkzeugkasten
- 3.5 Broadcasting und unser Semester-Operator
  - 3.5.1 Der Semester-Operator row_l2_normalize
  - 3.5.2 Compare: alle Skalarprodukte auf einmal
- 3.6 Die Suche in NumPy
  - 3.6.1 Acht Anfragen auf einmal
  - 3.6.2 Die ganze Suche am Stück
- 3.7 Sicht oder Kopie
- 3.8 Zum Weiterüben

3.1 Aufwärmen

Drei kurze Schnipsel aus dem Stoff von Grundkurs Teil 2. Sagen Sie jeweils die Ausgabe vorher und führen Sie die Zelle erst danach aus.

💭 Kurz überlegen

Was gibt die folgende Zelle aus?

```
xs = [3, 4, 5]
print([x * x for x in xs if x > 3])
```

💭 Kurz überlegen

Was gibt die folgende Zelle aus?

```
xs = [10, 20, 30, 40]
print(xs[1:-1])
```

💭 Kurz überlegen

Was gibt die folgende Zelle aus?

```
hits = [("Gehirn", 0.89), ("Zeit", 0.72)]
for title, score in hits:
    print(title, score)
```

3.2 Drei Fassungen einer Suche

Die nächsten beiden Zellen stellen die Kursdaten und die Funktionen aus der letzten Lerneinheit bereit: dieselben 400 Wikipedia-Absätze mit ihren Vektoren und dieselben acht Beispielanfragen. Führen Sie beide Zellen einfach aus, danach stehen die Daten und die Funktionen im Notebook bereit. Die Vektoren liegen diesmal in zwei Formen vor, als Python-Listen für die Fassungen in reinem Python und als NumPy-Arrays `E` und `Q` für die Fassung in NumPy. Ein Array ist der Zahlenblock, den `np.load` schon in den ersten beiden Lerneinheiten geliefert hat. Was genau darin steckt, ist das Thema des Kapitels 3.3. Die Funktionen tragen diesmal einen Docstring: den Text in dreifachen Anführungszeichen direkt unter der `def`-Zeile, der sagt, was die Funktion tut (eingeführt in Grundkurs Teil 2 beim Nachschlagen mit `help`). `norm?` zeigt ihn im Notebook an, und auch `help(norm)` liest ihn aus.

```python
# Vorgabe: Daten laden – einfach ausführen
import json, math, time
import numpy as np

with open("data/appetizer_passages.json", encoding="utf-8") as f:
    passages = json.load(f)["passages"]                    # 400 Absätze
with open("data/appetizer_queries.json", encoding="utf-8") as f:
    queries = json.load(f)["queries"]                      # 8 Beispielanfragen
E = np.load("data/appetizer_embeddings.npy")               # die Absatzvektoren als Array
Q = np.load("data/appetizer_query_embeddings.npy")         # die Anfragevektoren als Array
vectors = E.tolist()                                       # dieselben Zahlen als Listen
query_vectors = Q.tolist()
# Formen: E (400, 384), Q (8, 384); vectors ist eine Liste aus 400 Listen mit je 384 Zahlen

print(len(passages), "Absätze,", len(queries), "Beispielanfragen")
```

① `np.load` liest eine `.npy`-Datei als NumPy-Array; ohne `.tolist()` bleibt es ein Array

```
400 Absätze, 8 Beispielanfragen
```

Vertiefung: Das Dateiformat .npy

NICHT PRÜFUNGSRELEVANT

`.npy` ist das eigene Dateiformat von NumPy für ein einzelnes Array: ein kurzer Kopf mit Form und Datentyp, dahinter die Zahlen genau so, wie sie im Speicher liegen. `np.load` liest die Datei ohne Umwandlung ein, `np.save` schreibt sie.

Weil keine Textdarstellung dazwischenliegt, ist das Format schnell und verlustfrei: `float32` bleibt `float32`, und die 614.400 Bytes der Absatzvektoren ergeben eine Datei von 614.528 Bytes, der Rest ist der Kopf. JSON oder CSV müssten jede Zahl als Text schreiben und beim Lesen wieder in eine Zahl umwandeln. Mehrere Arrays in einer Datei speichert `np.savez` als `.npz`; das ist ein gewöhnliches ZIP-Archiv mit einer `.npy`-Datei je Array, das sich auch mit einem Packprogramm öffnen lässt. In der Projektaufgabe schreiben Sie selbst eine `.npy`-Datei.

```
# Vorgabe: Funktionen aus der letzten Lerneinheit – einfach ausführen
def norm(v):
    """Länge (Norm) des Vektors v: Wurzel aus der Summe der Quadrate."""
    total = 0.0
    for x in v:
        total += x * x
    return math.sqrt(total)

def normalize(v):
    """Neuer Vektor der Länge 1 mit derselben Richtung wie v."""
    length = norm(v)
    result = []
    for x in v:
        result.append(x / length)
    return result

def dot(a, b):
    """Skalarprodukt der Vektoren a und b (gleich viele Einträge)."""
    total = 0.0
    for i in range(len(a)):
        total += a[i] * b[i]
    return total

def measure(f, *args, repeats=3):
    """Führt f(*args) mehrfach aus und gibt die kürzeste gestoppte Zeit in Sekunden zurück."""
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        f(*args)
        times.append(time.perf_counter() - start)
    return min(times)

def scores_python(query_no):
    """Die Suche von Hand: Anfrage und alle Absätze normalisieren, 400 Skalarprodukte als Liste."""
    q = normalize(query_vectors[query_no])
    result = []
    for v in vectors:
        result.append(dot(q, normalize(v)))
    return result
```

`scores_python` ist die Suche aus der letzten Lerneinheit ohne die Auswahl der Treffer: Sie normalisiert die Anfrage und alle 400 Absatzvektoren und liefert die 400 Ähnlichkeitswerte. Alle drei Fassungen dieses Kapitels leisten genau diese Arbeit, damit die Messung fair und vergleichbar bleibt.

Grundkurs Teil 2 hat gezeigt, dass sich Schleifen oft kürzer schreiben lassen: `sum` summiert eine Liste, `zip` läuft über zwei Listen gleichzeitig, und eine List Comprehension baut eine neue Liste in einer Zeile. Die nächste Zelle zeigt die drei Werkzeuge an einem kleinen Beispiel.

```
a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]
① pairs = [(x, y) for x, y in zip(a, b)]
print(pairs)
② products = [x * y for x, y in zip(a, b)]
print(products, "summiert:", sum(products))
```

① `zip(a, b)` liefert Paare `(a[0], b[0])`, `(a[1], b[1])`, …, die Schleife packt jedes Paar in `x` und `y` aus
② die Comprehension baut aus jedem Paar ein Produkt, `sum` addiert die Liste

```
[(1.0, 4.0), (2.0, 5.0), (3.0, 6.0)]
[4.0, 10.0, 18.0] summiert: 32.0
```

💭 Kurz überlegen

Mit `sum`, `zip` und Comprehensions lassen sich `dot` und `normalize` in je einer Zeile schreiben. Nach der Aufgabe messen wir die ganze Suche in dieser Fassung gegen die Suche von Hand aus der letzten Lerneinheit. Ist die Fassung mit Bordmitteln schneller, langsamer, oder gleich schnell? Legen Sie sich fest, bevor Sie messen.

✏️ Aufgabe 3.2 #1 5 min

Schreiben Sie die Funktion `dot_builtin(a, b)`. Sie berechnet das Skalarprodukt der beiden Vektoren wie `dot`, aber mit `sum`, `zip` und einer List Comprehension in einer Zeile. Die Prüfzelle vergleicht beide Fassungen.

Wer früher fertig ist: Schreiben Sie in einer neuen Zelle auch `norm_builtin(v)` mit derselben Technik und prüfen Sie mit `abs(norm_builtin([3.0, 4.0]) - 5.0) < 1e-9`.

Hinweis: Die Zelle oben zeigt das Muster schon fast vollständig. Es fehlt nur noch das `return`.

```
def dot_builtin(a, b):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle — einfach ausführen
assert dot_builtin([1.0, 2.0, 3.0], [4.0, 5.0, 6.0]) is not None, "dot_builtin gibt noch nichts zurück (return vergessen?)"
assert abs(dot_builtin([1.0, 2.0, 3.0], [4.0, 5.0, 6.0]) - 32.0) < 1e-9, "1*4 + 2*5 + 3*6 = 32"
assert abs(dot_builtin(vectors[0], vectors[1]) - dot(vectors[0], vectors[1])) < 1e-9, "gleiches Ergebnis wie dot"
assert dot_builtin([], []) == 0, "zwei leere Vektoren ergeben 0"
print("Alle Prüfungen bestanden ✓")
```

Die zweite Fassung der Suche verwendet `dot_builtin` und eine ebenfalls mit Bordmitteln geschriebene Normalisierung. Die dritte Fassung rechnet in NumPy. Ihre beiden Zeilen müssen Sie noch nicht verstehen, am Ende dieser Lerneinheit haben Sie sie selbst geschrieben. Ein Unterschied bleibt und gehört dazugesagt: Die beiden Python-Fassungen rechnen mit Python-Zahlen (64 Bit), NumPy mit den gespeicherten 32-Bit-Zahlen.

```
# Vorgabe: die zweite und die dritte Fassung — einfach ausführen
def normalize_builtin(v):
    length = math.sqrt(sum([x * x for x in v]))
    return [x / length for x in v]

def scores_builtin(query_no):
    q = normalize_builtin(query_vectors[query_no])
    return [dot_builtin(q, normalize_builtin(v)) for v in vectors]

def scores_numpy(query_no):
    q = Q[query_no] / np.linalg.norm(Q[query_no])                # die Anfrage normalisieren
    # alle Absätze normalisieren, dann 400 Skalarprodukte auf einmal
    return (E / np.linalg.norm(E, axis=1, keepdims=True)) @ q
```

```
    t_python = measure(scores_python, 0)
    t_builtin = measure(scores_builtin, 0)
    t_numpy = measure(scores_numpy, 0, repeats=20)
    print(f"von Hand:        {t_python * 1000:7.2f} ms")
    print(f"mit Bordmitteln: {t_builtin * 1000:7.2f} ms")
    print(f"mit NumPy:       {t_numpy * 1000:7.2f} ms   Faktor {t_python / t_numpy:.0f} gegenüber der Fassung
von Hand")
```

```
von Hand:          12.83 ms
mit Bordmitteln:   15.40 ms
mit NumPy:          0.15 ms   Faktor 83 gegenüber der Fassung von Hand
```

Die Zahlen im Skript sind auf einem Referenzrechner gemessen, Ihre Werte weichen ab, die Größenordnung bleibt. In dieser Messung ist die Fassung mit Bordmitteln nicht schneller als die Schleife, und mehr als diese Beobachtung lässt sich aus einer Messung nicht ableiten. Die tragfähige Aussage ist eine andere: Die kürzere Schreibweise macht aus der Rechnung keine Array-Operation: Gerechnet wird weiter mit Python-Listen und einzelnen Zahlenobjekten, wie es das Laufzeitmikroskop in LE 2, Abschnitt 2.5.2 gezeigt hat. Den Unterschied macht erst NumPy, und wie es das tut, ist das Thema des nächsten Kapitels.

Vertiefung: Warum weichen die Ergebnisse von sum und der Schleife ab? NICHT PRÜFUNGSRELEVANT

Seit Python 3.12 summiert `sum` Fließkommazahlen mit einem Fehlerausgleich (dem Verfahren von Neumaier). Das Ergebnis ist genauer als das der einfachen Schleife und weicht von ihr in der letzten Stelle ab. Für die Suche spielt das keine Rolle, für Tests mit `==` schon.

Die Schleife addiert 384 Produkte nacheinander und rundet nach jeder Addition. `sum` führt dabei einen Korrekturterm mit, der die verlorenen Stellen sammelt. Die nächste Zelle zeigt die Abweichung an den Kursdaten. Mit einer Toleranz wie in den Prüfzellen ist sie unerheblich. Ein Test, der `dot` und `dot_builtin` mit `==` vergleicht, könnte dagegen scheitern.

```
    difference = abs(dot(vectors[0], vectors[1]) - dot_builtin(vectors[0], vectors[1]))
    print(f"Abweichung zwischen Schleife und sum: {difference:.1e}")
```

```
Abweichung zwischen Schleife und sum: 5.6e-17
```

Die Tabelle aus Listen. `vectors` ist eine Liste aus 400 Listen, also eine Tabelle mit 400 Zeilen und 384 Spalten. Eine Zeile liefert ein einzelner Zugriff, `vectors[0]`. Für eine Spalte gibt es keinen solchen Zugriff: Eine Comprehension muss alle 400 Zeilen durchlaufen und aus jeder den Eintrag an Position 5 herausholen, und jede weitere Spalte verlangt denselben Durchlauf noch einmal. Die Liste aus Listen speichert Zeilen; Spalten existieren in ihr nicht als eigene Objekte.

```
    first_row = vectors[0]                          # Zeile 0: der Vektor des ersten Absatzes
    column_5 = [row[5] for row in vectors]          # Spalte 5: Position 5 aus jedem der 400 Vektoren
    print(len(first_row), "Einträge in der Zeile,", len(column_5), "Einträge in der Spalte")
    print(f"Spalte 5 beginnt mit {column_5[0]:.4f}, {column_5[1]:.4f}, {column_5[2]:.4f}")
```

```
384 Einträge in der Zeile, 400 Einträge in der Spalte
Spalte 5 beginnt mit -0.0094, 0.1083, 0.0004
```

Für eine Rechnung über alle Spalten, etwa die Summe je Spalte, braucht die Tabelle aus Listen zwei verschachtelte Schleifen. NumPy behandelt Zeilen und Spalten dagegen gleichwertig: Beide sind Achsen eines zweidimensionalen Arrays, und jede Operation kann über jede Achse laufen.

3.3 Das Array: ein Block mit Beschreibung

Das Laufzeitmikroskop der letzten Lerneinheit endete mit einer Feststellung: Kompakter Speicher allein genügt nicht, erst zusammen mit einer Schleife in Maschinencode wird die Rechnung schnell. Genau diese Kombination ist ein NumPy-Array. Es besteht aus zwei Teilen: einem zusammenhängenden Block im Speicher, in dem die Zahlen dicht hintereinander liegen, und einer Beschreibung, wie dieser Block zu lesen ist. Die Beschreibung nennt die Form, den Datentyp und die Abstände zwischen den Einträgen. Elementweise Rechnungen und Zusammenfassungen wie Summen (sie heißen gleich Reduktionen) laufen als fertige, kompilierte Schleifen über diesen Speicherblock. Der Zugriff auf die Beschreibung (zum Beispiel `shape`) oder eine zweite Beschreibung desselben Blocks, etwa mit vertauschten Zeilen und Spalten, brauchen dagegen keinen Durchlauf durch die Werte. Dieses Kapitel sieht sich beide Teile an: zuerst den Block im Speicher, dann die Operationen, die die Beschreibung lesen oder ändern.

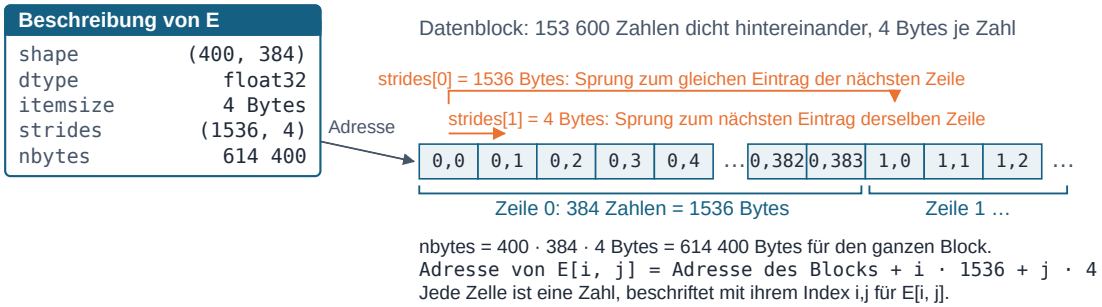


Die Python-Schleife trägt jede Zahl einzeln: Objekt holen, Typ prüfen, rechnen, Ergebnis ablegen. NumPy hebt den ganzen Block auf einmal, die Schleife läuft in kompiliertem Code über zusammenhängenden Speicher.

3.3.1 Wie die Zahlen im Speicher liegen

`E` ist die Matrix, die die Embedding-Vektoren unserer Kursdaten speichert, also der 400 Wikipedia-Absätze: 400 Zeilen, eine je Absatz, mit je 384 Zahlen für den Embedding-Vektor. Die Grafik zeigt, wie dieses Array im Speicher liegt, die Zelle darunter liest die Beschreibung aus, und die Tabelle danach fasst alle Attribute zusammen.

Ein Array: Datenblock und Beschreibung



Zum Vergleich die Liste aus Listen der letzten Lerneinheit: Verweise auf einzelne Zahlenobjekte zu je 24 Bytes, verstreut im Speicher, ohne gemeinsame Beschreibung.

```
  print(type(E))
① print("Form:", E.shape)
② print("Datentyp:", E.dtype, "mit", E.itemsize, "Bytes je Eintrag")
③ print("Achsen:", E.ndim, "  Einträge:", E.size, "  Bytes:", E.nbytes)
④ print("Abstände:", E.strides)
```

① `shape` ist die Form als Tupel: Anzahl der Zeilen, Anzahl der Spalten
② `dtype` ist der Datentyp aller Einträge, `itemsize` sein Speicherbedarf je Eintrag in Bytes
③ `ndim` ist die Anzahl der Achsen, `size` die Anzahl der Einträge, `nbytes` der Speicher für alle Einträge
④ `strides` sind die Abstände in Bytes zum nächsten Eintrag, je Achse einer

```
<class 'numpy.ndarray'>
Form: (400, 384)
Datentyp: float32 mit 4 Bytes je Eintrag
Achsen: 2   Einträge: 153600   Bytes: 614400
Abstände: (1536, 4)
```

Überblick: Die Beschreibung eines Arrays

Alle Angaben der Beschreibung lassen sich als Attribute abfragen. Die rechte Spalte zeigt die Werte für `E`.

Attribut	Bedeutung	Bei `E`
`shape`	Form: ein Tupel mit einer Länge je Achse	`(400, 384)`
`ndim`	Anzahl der Achsen, also die Länge von `shape`	`2`
`dtype`	der gemeinsame Datentyp des Arrays; bei unseren numerischen Arrays der Typ jedes einzelnen Zahlenwerts. `np.array([1, 2.5])` wird deshalb ganz zu `float64`	`float32`
`itemsize`	Bytes je Eintrag	`4`
`size`	Anzahl der Einträge, das Produkt der Längen in `shape`	`153600`
`nbytes`	Bytes für alle Einträge, `size` mal `itemsize`	`614400`
`strides`	Abstand in Bytes zum nächsten Eintrag, je Achse einer	`(1536, 4)`

Die Form ist ein Tupel, deshalb steht sie mit Klammern und Kommas: `(400, 384)` für zwei Achsen, `(384,)` für eine Achse mit 384 Einträgen (das Komma unterscheidet das Tupel von der eingeklammerten Zahl `(384)`) und `()` für eine einzelne Zahl ohne Achse. Beim Vergleichen von Formen, etwa in Prüfzellen, steht deshalb immer ein Tupel: `result.shape == (2,)`.

Die Abstände. Die Zahlen von `E` liegen zeilenweise hintereinander: erst die 384 Zahlen der Zeile 0, direkt dahinter die 384 Zahlen der Zeile 1 und so weiter. Um von `E[0, 0]` zu `E[0, 1]` zu kommen, geht NumPy 4 Bytes weiter, eine Zahl. Um von `E[0, 0]` zu `E[1, 0]` zu kommen, überspringt es eine ganze Zeile, 384 mal 4 = 1536 Bytes. Diese beiden Abstände sind die `strides` `(1536, 4)`. Mit ihnen berechnet NumPy aus jedem Indexpaar `(i, j)` die Speicheradresse des Eintrags: Adresse des Blocks plus `i * 1536 + j * 4`. Deshalb ist der Zugriff auf eine Spalte genauso einfach wie der auf eine Zeile, ein Ausdruck ohne Schleife und ohne Kopie, es ändert sich nur der Abstand. Ob das Lesen einer Spalte auch gleich schnell ist, hängt davon ab, wie die Zahlen im Speicher verteilt liegen (Kapitel 3.7). Und deshalb kann eine zweite Beschreibung denselben Block anders lesen, ohne eine Zahl zu kopieren: `E.T`, das transponierte `E` mit vertauschten Zeilen und Spalten (Kapitel 3.3.5), ist nichts anderes als `E` mit vertauschten Abständen `(4, 1536)`. Was daraus folgt, behandelt das Kapitel 3.7.

Die 614.400 Bytes sind die Zahl aus der Schätzaufgabe der letzten Lerneinheit: 400 mal 384 Zahlen zu je 4 Bytes. Als Python-Listen belegten dieselben Zahlen das Achtfache an Speicherplatz, weil dort jede Zahl ein eigenes Objekt ist und die Liste nur Verweise darauf enthält.

Zwei Bedeutungen von „Dimension". Ein Embedding-Vektor hat 384 Komponenten, man sagt auch, er ist 384-dimensional. Als Array liegt er trotzdem mit einer einzigen Achse vor, `ndim` ist 1: Bei „384-dimensional" meint Dimension die Anzahl der Komponenten, bei `ndim` die Anzahl der Achsen. Erst 400 Embedding-Vektoren untereinander ergeben zwei Achsen, die Form `(400, 384)`. Was eine Achse bedeutet, steht nirgends im Array. Es ergibt sich daraus, wie wir die Daten abgelegt haben: Beim Erzeugen der Datei wurde je Absatz eine Zeile geschrieben, deshalb zählt Achse 0 die Absätze und Achse 1 die

Komponenten. NumPy kennt diese Bedeutung nicht, es kennt nur zwei Achsen mit den Längen 400 und 384 und führt jede Operation auf der Achse aus, die der Code nennt. Die Verantwortung, die richtige Achse zu nennen, liegt beim Programm. Genau dafür steht in diesem Notebook nach jeder wichtigen Zelle die Formenzeile.

Ein Datentyp für alle Einträge. Anders als eine Python-Liste hat ein Array genau einen Datentyp, und zwar für alle Einträge zusammen; nur so kann NumPy jeden Eintrag über den Abstand finden, ohne nachzusehen, was dort liegt. Aus einer gemischten Liste wie `[1, "Text"]` macht `np.array` deshalb stillschweigend ein Array aus Zeichenketten, die 1 wird zu `"1"`. Verschiedene Typen nebeneinander gibt es auf zwei Wegen, die beide bei einem Typ je Array bleiben: Ein strukturiertes Array hat einen zusammengesetzten Datentyp mit benannten Feldern, etwa `dtype=[("nr", "i4"), ("titel", "U10"), ("score", "f4")]`; jeder Eintrag ist dann ein kleiner Datensatz mit festem Aufbau wie ein `struct` in C, und `a["titel"]` liefert die Spalte. Und ein Array vom Typ `object` enthält Verweise auf beliebige Python-Objekte, wie eine Liste, ohne den Geschwindigkeitsvorteil (Vertiefung zu den Datentypen in Kapitel 3.3.3). Für Tabellen mit Spalten verschiedenen Typs, etwa Titel und Zahlen nebeneinander, ist in der Praxis pandas die übliche Wahl (Kapitel 3.4).

Überblick: Wie NumPy aufgebaut ist

NumPy wird als `np` importiert und besteht aus wenigen Bausteinen, die in diesem Notebook immer wieder vorkommen. Die Beispiele in der Tabelle sind eine Vorschau; jedes wird in den folgenden Abschnitten eingeführt.

Baustein	Beispiele	Anmerkung
die Klasse `ndarray`	`E`, `Q`, mit `np.array` erzeugte Arrays, mit `np.load` aus `.npy`-Dateien geladene Arrays	das Array. Attribute wie `E.shape` beschreiben es, Methoden wie `E.sum()` rechnen auf ihm (bei `.npz` liefert `np.load` ein Archivobjekt, aus dem die Arrays einzeln entnommen werden)
Funktionen in `np`	`np.array(...)`, `np.zeros(...)`, `np.sqrt(x)`, `np.allclose(a, b)`	Arrays erzeugen, elementweise rechnen, vergleichen
Methode und Funktion mit derselben Aufgabe	`E.sum(axis=1)` und `np.sum(E, axis=1)`	viele Operationen gibt es in beiden Schreibweisen mit demselben Ergebnis. Für Reduktionen wie `sum` und `max` verwenden wir die Methode, für Werkzeuge wie `np.any`, `np.argsort` und `np.allclose` die Funktion
Unterpakete	`np.linalg.norm(E, axis=1)`, `np.random.default_rng(42)`	Funktionsgruppen mit eigenem Namensraum: lineare Algebra (`linalg`) und Zufallszahlen (`random`) kommen im Kurs vor, die übrigen zählt die Vertiefung unten auf
Datentypen	`np.float32`, `np.int64`	die Werte für `dtype=` und `astype`

Im Notebook zeigt `np.sum?` die Beschreibung einer Funktion an, wie in Grundkurs Teil 2 eingeführt. Die vollständige Dokumentation steht auf numpy.org.

Vertiefung: Die Unterpakete von NumPy

NICHT PRÜFUNGSRELEVANT

NumPy gliedert Funktionen für ein Fachgebiet in Unterpakete, die über `np.<name>.` erreichbar sind. Im Kurs brauchen Sie `np.linalg` und `np.random`; die anderen sollen Sie einordnen können, wenn sie Ihnen in fremdem Code begegnen.

Unterpaket	Inhalt	Beispiel
`np.linalg`	lineare Algebra: Normen, Determinanten, Gleichungssysteme lösen, Matrixzerlegungen	`np.linalg.norm(E, axis=1)`
`np.random`	Zufallszahlen aus verschiedenen Verteilungen mit festem Startwert (Seed)	`np.random.default_rng(42).standard_normal((N, D))`
`np.fft`	schnelle Fourier-Transformation: ein Signal in seine Frequenzanteile zerlegen (Audio, Bilder)	`np.fft.fft(signal)`
`np.polynomial`	Polynome anlegen, auswerten, an Daten anpassen	`np.polynomial.Polynomial.fit(x, y, 2)`
`np.ma`	maskierte Arrays: Rechnen mit fehlenden Werten	`np.ma.masked_invalid(x).mean()`
`np.testing`	Vergleichsfunktionen für Tests, mit lesbaren Fehlermeldungen	`np.testing.assert_allclose(a, b)`
`np.lib.stride_tricks`	Sichten mit frei gewählten Abständen, etwa gleitende Fenster	`sliding_window_view(x, 3)`

Was nicht in NumPy steckt, liefert SciPy auf derselben Array-Basis: Optimierung, Statistik, Signalverarbeitung, dünn besetzte Matrizen (Kapitel 3.4).

3.3.2 Vom Einzelwert zum Tensor

Das Wort „Tensor" steht im Namen dieses Kurses und im Namen der wichtigsten Klasse von PyTorch. Bevor es um das Rechnen mit Arrays geht, gehört deshalb geklärt, was ein Tensor ist und warum NumPy-Arrays schon welche sind.

Definition: Tensor

Ein Tensor ist eine Anordnung von Zahlen gleichen Typs, in der jeder Eintrag durch ein festes Tupel von Indizes bestimmt ist. Die Anzahl der Indizes heißt Stufe (auch Ordnung; in manchen Texten „Rang", gemeint ist dann die Anzahl der Achsen, nicht der Matrixrang aus der linearen Algebra) des Tensors: Ein Skalar (eine einzelne Zahl) hat die Stufe 0, ein Vektor die Stufe 1, eine Matrix die Stufe 2. In NumPy ist ein Tensor der Stufe k ein Array mit `ndim == k`; seine Form gibt für jeden Index an, wie viele Werte er annehmen kann.

In der Schreibweise der Mathematik: Ein Tensor der Stufe k mit der Form $(n_1, \ldots, n_k)$ ist eine Abbildung, die jedem Indextupel eine reelle Zahl zuordnet,

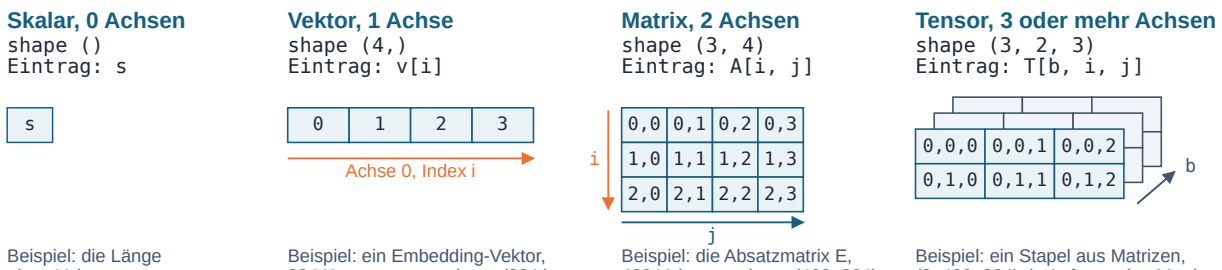
$$T \in \mathbb{R}^{n_1 \times n_2 \times \ \ldots \ \times n_k}, \qquad T_{i_1 i_2 \ldots i_k} \in \mathbb{R}$$

mit $0 \le i_m < n_m$ für jeden Index m. Die Mathematik schreibt die Indizes tiefgestellt, NumPy in eckigen Klammern: `T[i_1, i_2, ..., i_k]`. Die Stufen im Einzelnen:

- **Skalar, Stufe 0:** eine einzelne Zahl, Form `()`. Beispiel: die Länge eines Vektors.
- **Vektor, Stufe 1:** Zahlen entlang einer Achse, der Eintrag `v[j]` ist durch einen Index bestimmt. Beispiel: ein Embedding-Vektor, Form `(384,)`.
- **Matrix, Stufe 2:** Zahlen in Zeilen und Spalten, der Eintrag `E[i, j]` braucht zwei Indizes. Beispiel: die Absatzmatrix, Form `(400, 384)`.
- **Tensor der Stufe 3:** ein Stapel aus Matrizen, der Eintrag `T[b, i, j]` braucht drei Indizes. Beispiel: je eine Matrix pro Anfrage, Form `(8, 400, 384)`.
- **Höhere Stufen:** so geht es weiter. Ein Stapel aus Farbbildern hat beispielsweise die Form `(Anzahl, Höhe, Breite, Farbkanäle)`, vier Indizes.

In der Physik und der Differentialgeometrie gehört zur Definition zusätzlich, wie sich die Einträge bei einem Wechsel des Koordinatensystems verhalten. Davon brauchen die KI-Bibliotheken nichts: Dort heißt jedes mehrdimensionale Array Tensor, in NumPy `ndarray`, in PyTorch `Tensor`, und gemeint ist dasselbe Objekt aus Block und Beschreibung. Das Denken in Formen und Achsen, das Sie in dieser Lerneinheit lernen, lässt sich unverändert auf PyTorch übertragen; bei einzelnen Funktionen prüfen wir dort Parameter und Voreinstellungen erneut, denn `std` etwa rechnet in NumPy und PyTorch mit verschiedenen Voreinstellungen.

Vom Einzelwert zum Tensor: ein Index je Achse



Jede Zelle ist mit ihrem Index beschriftet: ein Index je Achse. Jede Stufe fügt eine Achse und damit einen Index hinzu. In NumPy sind alle vier dasselbe Objekt, ein ndarray; sie unterscheiden sich nur in ndim und shape.

In NumPy ist diese Stufenleiter kein Unterschied in der Art des Objekts. Vektor, Matrix und höherer Tensor sind dasselbe `ndarray`, sie unterscheiden sich nur in `ndim` und `shape`; selbst eine einzelne Zahl lässt sich mit `np.array(5.0)` als Array mit null Achsen anlegen. Attribute wie `shape` und `dtype` und die elementweisen Rechenregeln gelten auf allen Stufen gleich. Operationen mit Achsenangabe oder das Matrixprodukt brauchen dagegen passende Formen, ein Vektor hat zum Beispiel keine Achse 1. Die nächste Zelle baut die vier Stufen aus den Kursdaten. Ein Einzelzugriff wie `E[0, 0]` liefert übrigens kein Array, sondern einen NumPy-Skalar vom Typ `np.float32`; er trägt dieselben Attribute, deshalb zeigt die Zelle den Unterschied mit `type`.

```
① scalar = np.array(E[0, 0])       # eine einzelne Zahl aus E, als Array der Stufe 0
  vector = E[0]                    # Zeile 0: der Embedding-Vektor des ersten Absatzes
  matrix = E                       # alle 400 Vektoren
② stack = np.stack([E, E])         # zwei Kopien von E übereinander: Stufe 3
  for name, arr in [("Skalar", scalar), ("Vektor", vector), ("Matrix", matrix), ("Tensor Stufe 3", stack)]:
      print(f"{name:15s} ndim = {arr.ndim}   shape = {str(arr.shape):15s} {type(arr).__name__}")
  print(type(E[0, 0]).__name__, "<- der Einzelzugriff liefert einen NumPy-Skalar, kein Array")
  # Formen: np.array(E[0, 0]) ();  E[0] (384,);  E (400, 384);  np.stack([E, E]) (2, 400, 384)
```

① `np.array(zahl)` legt ein Array mit null Achsen an; `E[0, 0]` allein wäre ein NumPy-Skalar (`np.float32`)
② `np.stack` legt gleich geformte Arrays entlang einer neuen ersten Achse übereinander

```
Skalar          ndim = 0   shape = ()              ndarray
Vektor          ndim = 1   shape = (384,)          ndarray
Matrix          ndim = 2   shape = (400, 384)      ndarray
Tensor Stufe 3  ndim = 3   shape = (2, 400, 384)   ndarray
float32 <- der Einzelzugriff liefert einen NumPy-Skalar, kein Array
```

„Tensor Thinking": Rechnungen über Formen planen, bevor Code entsteht. Statt zu fragen „Welche Schleife läuft über welchen Index?", fragen Sie „Welche Form hat die Eingabe, welche Form soll das Ergebnis haben, und welche Operation führt von der einen zur anderen?". Die Schleifen stecken dann in den Achsen.

Dieser Denkschritt zieht sich durch die ganze Lerneinheit, und die Formenzeile hält ihn in jeder Zelle fest. Aus „für jeden Absatz die Länge seines Vektors" wird `E (400, 384) -> (400,)`: eine Operation, die die zweite Achse zusammenfasst (sie heißt gleich Reduktion). Aus „die Ähnlichkeit jeder Anfrage zu jedem Absatz" wird `(8, 384) @ (384, 400) -> (8, 400)`: eine einzige Operation statt zweier Schleifen, der Operator `@` kommt im Kapitel 3.5. In PyTorch planen Sie später jede Schicht eines neuronalen Netzes genau so, über die Formen der Tensoren, die hinein- und herausgehen.

3.3.3 Arrays anlegen, Zugriff und Slicing

Arrays anlegen. Die Kursdaten kommen fertig aus `np.load`. Eigene Arrays brauchen Sie trotzdem laufend: für die kleinen Testfälle der Prüfzellen, für Zufallsdaten beim Messen und für Ergebnisse, die Stück für Stück gefüllt werden. `np.array` baut aus einer Liste ein Array und wählt den Datentyp nach den Werten: Ganzzahlen ergeben `int64`, Fließkommazahlen `float64`. Mit `dtype=` legen Sie den Typ selbst fest. Die Kursdaten sind ausdrücklich als `float32` gespeichert, das halbiert den Speicher gegenüber `float64` und ist in KI-Bibliotheken der übliche Typ. `astype` wandelt ein vorhandenes Array in einen anderen Typ um, zum Beispiel Zufallszahlen, die als `float64` entstehen, in das `float32`, das der Operator verlangt. `np.zeros` legt ein Array mit Nullen in einer gewünschten Form an, `np.arange` zählt wie `range`.

```
  print(np.array([1, 2]).dtype, np.array([1.0, 2.0]).dtype)
① x = np.array([[3.0, 4.0], [0.0, 2.0]], dtype=np.float32)       # zwei Zeilen, zwei Spalten
  print(x.shape, x.dtype)
② print(np.array([1, 2]).astype(np.float32).dtype)
③ print(np.zeros((2, 3), dtype=np.float32))
④ print(np.arange(6))
```

① `dtype=np.float32` legt den Typ beim Anlegen fest
② `astype` liefert ein neues Array im gewünschten Typ
③ `np.zeros((2, 3), dtype=...)` legt ein Array der Form (2, 3) mit Nullen an, die Form ist ein Tupel
④ `np.arange(6)` zählt von 0 bis 5 wie `range(6)`, liefert aber ein Array

```
int64 float64
(2, 2) float32
float32
[[0. 0. 0.]
 [0. 0. 0.]]
[0 1 2 3 4 5]
```

Überblick: Arrays anlegen

Aufruf	Was entsteht	Beispielergebnis
`np.array([[3., 4.], [0., 2.]], dtype=np.float32)`	Array aus einer Liste von Listen, Typ ausdrücklich	`[[3., 4.], [0., 2.]]`, Form `(2, 2)`, `float32`
`np.zeros((2, 3), dtype=np.float32)`	Nullen in der angegebenen Form; `np.ones` entsprechend mit Einsen	`[[0., 0., 0.], [0., 0., 0.]]`
`np.arange(6)`	Ganzzahlen von 0 bis 5 wie `range`	`[0, 1, 2, 3, 4, 5]`
`np.linspace(0, 1, 5)`	fünf gleichmäßig verteilte Werte von 0 bis 1 einschließlich	`[0., 0.25, 0.5, 0.75, 1.]`
`np.load("data/x.npy")`	Array aus einer `.npy`-Datei, Form und Typ aus der Datei	`E` mit `(400, 384)`, `float32`
`rng = np.random.default_rng(42)`, `rng.standard_normal((2, 3))`	Zufallszahlen mit festem Startwert, Typ `float64`	`[[0.30, -1.04, 0.75], [0.94, -1.95, -1.30]]`
`x.astype(np.float32)`	dieselben Werte in einem anderen Typ, standardmäßig als neues Array	aus `[1, 2]` (`int64`) wird `[1., 2.]` (`float32`)

Vertiefung: Die Datentypen von NumPy

Im Kurs genügen `float32` für Daten und Rechnungen, `float64` als Python-Standard, `int64` für Positionen und `bool` für Masken. NumPy kennt darüber hinaus eine ganze Familie von Typen mit fester Bitbreite, wie Sie sie aus C kennen.

Typ	Speicher je Eintrag	Wertebereich bzw. Genauigkeit	Typischer Einsatz
`bool`	1 Byte	`True`, `False`	Masken
`int8`, `int16`, `int32`, `int64`	1, 2, 4, 8 Bytes	ganze Zahlen mit Vorzeichen, `int8` von −128 bis 127	Positionen (Indexarrays verwenden `np.intp`, auf unseren 64-Bit-Systemen `int64`), Zähler, Pixelwerte
`uint8` … `uint64`	1 bis 8 Bytes	ganze Zahlen ohne Vorzeichen, `uint8` von 0 bis 255	Bilddaten (ein Farbkanal je Byte)
`float16`	2 Bytes	10 Mantissenbits, rund 3 signifikante Dezimalstellen, kleiner Exponentenbereich	Inferenz auf GPUs, Speicher sparen
`bfloat16`	2 Bytes	7 Mantissenbits, rund 2 signifikante Dezimalstellen, dafür der Exponentenbereich von `float32`	Training mit gemischter Genauigkeit; in PyTorch eingebaut, in NumPy nur über Erweiterungen wie `ml_dtypes`
`float32`	4 Bytes	rund 7 signifikante Dezimalstellen	Standard in KI-Bibliotheken, unsere Kursdaten
`float64`	8 Bytes	rund 16 signifikante Dezimalstellen	Python-`float`, wissenschaftliches Rechnen, Standard von `np.array([1.0])`
`complex64`, `complex128`	8, 16 Bytes	komplexe Zahlen	Signalverarbeitung (`np.fft`)
`str_`	fest je Array, z. B. 12 Bytes je Eintrag bei `dtype="U3"` (3 Zeichen zu 4 Bytes)	Zeichenketten bis zu einer festen Länge	selten, Text gehört zu pandas
`object`	8 Bytes je Verweis, die Objekte selbst liegen außerhalb und zählen in `nbytes` nicht mit	beliebige Python-Objekte, auch gemischte Typen: `np.array([1, "Text"], dtype=object)`	ein Feld von Verweisen wie eine Liste, ohne den Geschwindigkeitsvorteil

Ganzzahltypen laufen bei Überschreitung des Wertebereichs stillschweigend über: `np.array([127], dtype=np.int8) + 1` ergibt `-128`. Bei Rechnungen zwischen zwei NumPy-Typen bestimmt NumPy den Ergebnistyp nach festen Regeln (Promotion): `int64` plus `float32` ergibt `float64`, wobei sehr große Ganzzahlen dabei ihre exakte Darstellung verlieren können. Für Python-Zahlen gilt: Ein Python-`float` lässt ein `float32`-Array bei `float32`, hebt ein Ganzzahlarray aber auf `float64` an; ein Python-`int` passt sich einem Ganzzahlarray an (Kapitel 3.5). Die Grenzen eines Typs zeigen `np.finfo` für Fließkommatypen und `np.iinfo` für Ganzzahltypen an, zum Beispiel `np.finfo(np.float32)` und `np.iinfo(np.int8)`.

Genau verstehen: astype kopiert standardmäßig

`astype` legt in der Voreinstellung ein neues Array an, auch wenn der Zieltyp dem alten gleicht. Der Aufruf kostet also Speicher und Zeit in der Größe des Arrays, bei 100.000 Zeilen mit 384 Spalten rund 150 MB und spürbar Zeit. In Rechnungen, die oft laufen, legt man den Typ deshalb einmal beim Laden fest und wandelt danach nicht mehr um. Die Kursdaten sind aus diesem Grund als `float32` gespeichert, und der Operator verlangt `float32` statt stillschweigend umzuwandeln. Mit `astype(np.float32, copy=False)` gibt NumPy das vorhandene Array zurück, wenn der Typ schon passt, und kopiert nur im anderen Fall.

Zugriff und Slicing. Bei einer Liste steht in den eckigen Klammern ein Index oder ein Slice. Bei einem Array steht dort ein Eintrag je Achse, durch Kommas getrennt: vor dem Komma die Zeilen, dahinter die Spalten. Für die Absatzmatrix `E` heißt das: `E[108]` ist der Vektor des Absatzes 108, eine Zeile. `E[108, 5]` ist seine Komponente 5, ein einzelner Eintrag. `E[:, 5]` ist die Komponente 5 aller 400 Absätze, eine Spalte; `:` allein heißt „alle Einträge dieser Achse". Der Zugriff auf eine Spalte ist damit genauso kurz geschrieben wie der auf eine Zeile, NumPy berechnet beide aus den Abständen. Für die Form des Ergebnisses gilt eine Regel, die Sie oft brauchen werden: Ein einzelner Index wählt einen Eintrag der Achse aus, und die Achse fällt aus der Form weg. Ein Slice wählt einen Bereich aus, und die Achse bleibt mit der Länge des Bereichs erhalten. `E[0]` hat deshalb die Form `(384,)` und `E[0:1]` die Form `(1, 384)`, obwohl beide dieselben 384 Zahlen enthalten. Der Unterschied spielt eine Rolle, sobald zwei Arrays verschiedener Form miteinander verrechnet werden (Kapitel 3.5).

Zum Nachvollziehen mit Werten verwenden die nächsten Abschnitte neben `E` eine kleine Matrix `M` mit den Zahlen 1 bis 12 in drei Zeilen und vier Spalten. An ihr lässt sich jedes Ergebnis im Kopf prüfen.

> 💭 **Kurz überlegen**
>
> Welche Form haben `E[5]`, `E[5:6]`, `E[:, :10]` und `E[::2]`? Begründen Sie jede Form mit der Regel „Index entfernt die Achse, Slice behält sie", bevor Sie die nächste Zelle ausführen.

```
  M = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])          # Form (3, 4), zum Nachvollziehen
① print(M[0], M[0].shape)                  # Zeile 0: der Index entfernt die Achse
  print(M[0:1], M[0:1].shape)              # Zeile 0 als Slice: die Achse bleibt
  print(M[:, 1])                           # Spalte 1: ein Eintrag je Zeile
  print(M[1:, :2])                         # ab Zeile 1, die ersten zwei Spalten
  print(E[0].shape, E[0:1].shape, E[:, 5].shape, E[:3, :2].shape)   # dasselbe an den Kursdaten
  # Formen: M[0] (4,);  M[0:1] (1, 4);  M[:, 1] (3,);  M[1:, :2] (2, 2)
  #         E[0] (384,);  E[0:1] (1, 384)
```

① vor dem Komma die Zeilen, dahinter die Spalten; `:` allein heißt „alle"

```
[1 2 3 4] (4,)
[[1 2 3 4]] (1, 4)
[ 2  6 10]
[[ 5  6]
 [ 9 10]]
(384,) (1, 384) (400,) (3, 2)
```

Überblick: Zugriff und Slicing

An `M` mit den Werten 1 bis 12 in der Form `(3, 4)`; für `E` mit `(400, 384)` gelten dieselben Regeln. Negative Indizes zählen wie bei Listen vom Ende her.

Ausdruck	Bedeutung	Ergebnis an `M`	Form
`M[0]`	Zeile 0, der Index entfernt die Achse	`[1 2 3 4]`	`(4,)`
`M[0:1]`	Zeile 0 als Slice, die Achse bleibt	`[[1 2 3 4]]`	`(1, 4)`
`M[0, 2]`	ein Eintrag: Zeile 0, Spalte 2	`3`	`()`
`M[:, 1]`	Spalte 1, ein Eintrag je Zeile	`[2 6 10]`	`(3,)`
`M[1:, :2]`	ab Zeile 1, Spalten 0 und 1	`[[5 6] [9 10]]`	`(2, 2)`
`M[-1]`	die letzte Zeile	`[9 10 11 12]`	`(4,)`
`M[::2]`	jede zweite Zeile	`[[1 2 3 4] [9 10 11 12]]`	`(2, 4)`

Ein Slice liefert bei Arrays keine neue Liste wie in Grundkurs Teil 2, sondern eine Sicht (View) auf denselben Datenblock. Was das bedeutet, zeigt das Kapitel 3.7.

3.3.4 Achsen und Reduktionen

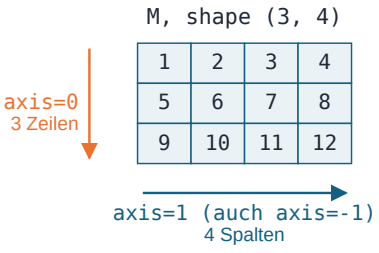
Eine Reduktion fasst viele Werte zu einem zusammen: `sum` addiert, `max` nimmt den größten, `argmax` die Position des größten, `mean` bildet den Mittelwert. Für die Suche brauchen Sie Reduktionen an zwei Stellen: Die Länge eines Vektors ist die Wurzel aus der Summe seiner Quadrate, also eine Summe über die Komponenten, und das Skalarprodukt ist die Summe über die Produkte zweier Vektoren. Bei einer Matrix kommt die Frage dazu, in welche Richtung summiert wird: alle Einträge zu einer Zahl, die Zeilen aufeinander oder die Spalten nebeneinander. Das legt `axis=` fest. An `M` lässt sich jedes Ergebnis nachrechnen.

```
   print(M.sum(), M.sum().shape)           # alle Einträge: eine Zahl, Form ()
①  print(M.sum(axis=0))                    # Zeilen aufeinander: [1+5+9, 2+6+10, 3+7+11, 4+8+12]
   print(M.sum(axis=1))                    # Spalten nebeneinander: [1+2+3+4, 5+6+7+8, 9+10+11+12]
②  print(M.sum(axis=1, keepdims=True))     # dieselben drei Werte als Spalte, Form (3, 1)
③  print(M.max(axis=1), M.argmax(axis=1))  # je Zeile: der größte Wert und seine Spalte
   # Formen: M (3, 4) -> sum() ();  M (3, 4) -> sum(axis=0) (4,)
   #         M (3, 4) -> sum(axis=1) (3,);  M (3, 4) -> sum(axis=1, keepdims=True) (3, 1)
```

① `axis=0` summiert entlang der Achse 0, `axis=1` entlang der Achse 1
② `keepdims=True` behält die summierte Achse mit der Länge 1
③ `argmax` liefert die Position des größten Werts, `max` den Wert selbst

```
78 ()
[15 18 21 24]
[10 26 42]
[[10]
 [26]
 [42]]
[ 4  8 12] [3 3 3]
```

Achsen und axis= bei einer Reduktion, am Beispiel M mit shape (3, 4)



Der Index i zählt entlang Achse 0,
der Index j entlang Achse 1: M[i, j].

`M.sum(axis=0)`
addiert entlang Achse 0, also die Zeilen aufeinander. Achse 0 verschwindet.

15	18	21	24

`shape (4,) ein Wert je Spalte`

`M.sum(axis=1)`
addiert entlang Achse 1, also die Spalten nebeneinander. Achse 1 verschwindet.

10	26	42

`shape (3,) ein Wert je Zeile`

`M.sum(axis=1, keepdims=True)`
dieselben Werte, die Achse 1 bleibt mit Länge 1: eine Spalte

10
26
42

`shape (3, 1) passt neben jede Zeile von M`

Merkregel: Die Achse, die bei axis= steht, wird zusammengefasst und verschwindet aus der Form.
Mit keepdims=True bleibt sie mit der Länge 1 stehen. Für E gilt dasselbe mit (400, 384) statt (3, 4).

Warum es darauf ankommt, ob die Achse bleibt. `M.sum(axis=1)` und `M.sum(axis=1, keepdims=True)` enthalten dieselben drei Zahlen. Der Unterschied ist die Form, `(3,)` gegen `(3, 1)`, und die entscheidet darüber, wie das Ergebnis weiterverwendet werden kann. Ohne `keepdims` ist das Ergebnis ein gewöhnlicher Vektor mit einem Wert je Zeile: die richtige Form, um die Werte auszugeben, zu sortieren oder als Trefferliste zu verwenden. Mit `keepdims=True` behält das Ergebnis die Gestalt einer Spalte neben der Matrix: die richtige Form, wenn jede Zeile der Matrix mit „ihrem" Wert weiterverrechnet werden soll, etwa jede Zeile durch ihre eigene Länge geteilt. Warum dafür die Spaltenform nötig ist und was ohne sie geschieht, zeigt das Kapitel 3.5 mit den Regeln für das Verrechnen verschiedener Formen. Bis dahin genügt die Faustregel: Ergebnis zum Ausgeben ohne `keepdims`, Ergebnis zum Weiterrechnen mit der Matrix mit `keepdims=True`.

Auf den Kursdaten sehen dieselben Aufrufe genauso aus, nur die Formen sind größer. `axis=-1` bezeichnet die letzte Achse, bei zwei Achsen also dieselbe wie `axis=1`. Die Schreibweise ist in PyTorch-Code üblich, weil sie bei unseren Daten unabhängig von der Anzahl der Achsen „über die Komponenten" meint: Bei `E` mit `(400, 384)` und bei einem Stapel mit

`(8, 400, 384)` liegen die Komponenten jeweils auf der letzten Achse. Das gilt nur, weil die Daten so angeordnet sind. Bei `E.T` mit der Form `(384, 400)` liegen auf der letzten Achse die Absätze, dort würde `axis=-1` über die Absätze summieren.

```
  print(E.sum(axis=0).shape, E.sum(axis=1).shape, E.sum(axis=1, keepdims=True).shape)
① print(E.sum(axis=-1).shape)
  print(E.max(axis=1)[:3], E.argmax(axis=1)[:3])
  # Formen: E (400, 384) -> sum(axis=0) (384,);  E (400, 384) -> sum(axis=1) (400,)
  #         E (400, 384) -> sum(axis=1, keepdims=True) (400, 1)
```

① `axis=-1` ist die letzte Achse, hier dieselbe wie `axis=1`

```
(384,) (400,) (400, 1)
(400,)
[0.39116466 0.4714218  0.46661583] [296  33 334]
```

Überblick: Reduktionen und Achsen

Für `E` mit der Form `(400, 384)`.

Aufruf	Ergebnis	Form
`E.sum()`	Summe aller Einträge, eine Zahl	`()`
`E.sum(axis=0)`	ein Wert je Spalte, die Zeilen aufeinander addiert	`(384,)`
`E.sum(axis=1)`	ein Wert je Zeile, die Spalten nebeneinander addiert	`(400,)`
`E.sum(axis=-1)`	dasselbe, `-1` bezeichnet die letzte Achse	`(400,)`
`E.sum(axis=1, keepdims=True)`	ein Wert je Zeile, die Achse bleibt mit Länge 1	`(400, 1)`
`E.max(axis=1)`, `E.argmax(axis=1)`	größter Wert je Zeile, seine Position je Zeile	`(400,)`
`E.mean()`, `E.std()`, `E.min()`	weitere Reduktionen, mit und ohne `axis=`	wie `sum`

Merkregel: Die bei `axis=` genannte Achse wird zusammengefasst. Ohne `keepdims` verschwindet sie aus der Form, mit `keepdims=True` bleibt sie mit der Länge 1 erhalten.

Vertiefung: Welche Reduktionen gibt es noch?

Alle Reduktionen von NumPy nehmen `axis=` und `keepdims=` auf dieselbe Weise an. Neben `sum`, `max` und `argmax` brauchen Sie im Kurs vor allem `mean` und `std`, die in der nächsten Lerneinheit bei der Datenanalyse vorkommen.

Reduktion	Ergebnis	An `M` mit `axis=1`
`M.mean(axis=1)`	arithmetischer Mittelwert, Summe geteilt durch die Anzahl	`[2.5, 6.5, 10.5]`
`M.std(axis=1)`	Standardabweichung: die Wurzel aus dem Mittelwert der quadrierten Abstände zum Mittelwert, ein Maß für die Streuung	`[1.118, 1.118, 1.118]`
`M.min(axis=1)`, `M.argmin(axis=1)`	kleinster Wert und seine Position	`[1, 5, 9]`, `[0, 0, 0]`
`M.prod(axis=1)`	Produkt aller Einträge	`[24, 1680, 11880]`
`(M > 6).any(axis=1)`	mindestens ein Eintrag der Zeile ist wahr	`[False, True, True]`
`(M > 6).all(axis=1)`	alle Einträge der Zeile sind wahr	`[False, False, True]`
`M.cumsum(axis=1)`	laufende Summe; keine Reduktion im engen Sinn, die Achse bleibt	`[[1, 3, 6, 10], [5, 11, 18, 26], [9, 19, 30, 42]]`

`np.linalg.norm(M, axis=1)` ist ebenfalls eine Reduktion über eine Achse: die Wurzel aus der Summe der Quadrate, genau das, was die Funktion `row_norms` im Kapitel 3.3.6 berechnet.

> 💭 **Kurz überlegen**
>
> Welche Form haben `Q.sum(axis=0)`, `Q.max(axis=1)` und `Q.sum(axis=1, keepdims=True)`? `Q` hat die Form `(8, 384)`. Schreiben Sie die drei Formen auf, bevor Sie die nächste Zelle ausführen.

```
print(Q.sum(axis=0).shape, Q.max(axis=1).shape, Q.sum(axis=1, keepdims=True).shape)
```

3.3.5 Formen ändern: Transponieren und Umformen

Transponieren. Die Transponierte einer Matrix vertauscht Zeilen und Spalten: Aus `A` mit der Form `(m, n)` wird `A.T` mit der Form `(n, m)`, und für jeden Eintrag gilt `A.T[j, i] == A[i, j]`. Gebraucht wird das vor allem, um Formen für das Matrixprodukt passend zu machen: Die Ähnlichkeit aller Anfragen zu allen Absätzen ist später `Qn @ En.T`, mit `Qn` und `En` als zeilenweise normalisierten Fassungen von `Q` und `E`, weil dort die 384 Komponenten der Anfragen auf die 384 Komponenten der Absätze treffen müssen. Außerdem macht `.T` aus den Spalten einer Matrix Zeilen, wenn eine Operation zeilenweise arbeitet und man sie auf die Spalten anwenden möchte. Bei einem Array mit einer Achse ändert `.T` nichts, es gibt keine zweite Achse zum Tauschen.

Transponieren in mathematischer Schreibweise

Für eine Matrix A mit m Zeilen und n Spalten ist die Transponierte $A^{\top}$ die Matrix mit n Zeilen und m Spalten, deren Einträge

$$(A^{\top})_{ji} = A_{ij}$$

sind, für alle Zeilen i von 0 bis m−1 und alle Spalten j von 0 bis n−1. Aus der Form `(m, n)` wird `(n, m)`. Ein Sonderfall ist der Zeilenvektor, eine Matrix mit nur einer Zeile: Aus `(1, n)` wird der Spaltenvektor `(n, 1)` und umgekehrt.

Umformen. `reshape` gibt denselben Einträgen eine andere Form. Standardmäßig liest es die Einträge in logischer Zeilenreihenfolge, also Zeile für Zeile von links nach rechts, und ordnet sie in dieser Reihenfolge in die neue Form ein. Zwei Regeln: Die Anzahl der Einträge muss gleich bleiben, aus 6 Einträgen werden `(2, 3)`, `(3, 2)` oder `(6,)`, aber nicht `(4, 2)`; dann bricht NumPy mit `ValueError: cannot reshape array of size 6 into shape (4,2)` ab. Und eine `-1` steht für „diese Länge ausrechnen", höchstens eine Länge darf so offen bleiben. Typische Anwendungen: ein Bild aus `(28, 28)` Pixeln als Vektor der Form `(784,)` an ein neuronales Netz übergeben, einen Stapel `(8, 400, 384)` als `(3200, 384)` zeilenweise verarbeiten, eine Achse der Länge 1 einfügen. Dieselbe Form heißt nicht dieselbe Anordnung: `a.reshape(3, 2)` und `a.T` eines `(2, 3)`-Arrays haben beide die Form `(3, 2)`, enthalten die Werte aber in anderer Reihenfolge. `reshape` liest die Einträge der Reihe nach und bricht sie neu um, `.T` spiegelt an der Diagonalen. Ob `reshape` dafür Daten kopieren muss, hängt davon ab, wie das Array im Speicher liegt: Sicht, wenn möglich, sonst Kopie. Für das zusammenhängende `a` unten ist `a.reshape(-1)` eine Sicht: Die Werte `0, 1, 2, 3, 4, 5` liegen im Block schon in dieser Reihenfolge, die neue Form braucht nur neue Abstände. `a.T.reshape(-1)` liefert dagegen `[0, 3, 1, 4, 2, 5]`, und diese Reihenfolge gibt es im Block nicht: Die Werte 0 und 3 liegen drei Plätze auseinander, 3 und 1 zwei Plätze rückwärts. Kein fester Abstand beschreibt das, also legt NumPy einen neuen Block an. Am Ergebnis ändert das nichts, die Werte sind dieselben; der Unterschied zeigt sich im Speicherbedarf und darin, ob Schreibzugriffe das Original treffen (Kapitel 3.7). Muss man als Entwickler also ständig darauf achten? Beim Lesen nicht, die Werte sind in beiden Fällen dieselben. Es zählt in zwei Situationen: wenn Sie in das Ergebnis schreiben, denn eine Sicht ändert das Original mit, und wenn das Array groß ist, denn eine Kopie kostet Speicher und Zeit. Wer sicher sein will, verlangt mit `.copy()` ausdrücklich eine Kopie. Die Regeln dazu stellt das Kapitel 3.7 zusammen.

> 💭 **Kurz überlegen**
>
> `a = np.arange(6).reshape(2, 3)` enthält die Werte 0 bis 5 in zwei Zeilen. Welcher Wert steht in `a.T[1, 0]`, welcher in `a.reshape(3, 2)[1, 0]`? Rechnen Sie beide aus, bevor Sie die nächste Zelle ausführen.

```
① a = np.arange(6).reshape(2, 3)
  print(a)
② print(a.reshape(3, -1))                # die Werte zeilenweise neu umgebrochen
  print(a.T)                             # vertauscht: gleiche Form, andere Anordnung
  print(a.reshape(-1), a.reshape(-1).shape)   # flach: eine Achse mit allen Einträgen
  print(E.T.shape)
  # Formen: a (2, 3) -> reshape(3, -1) (3, 2);  a (2, 3) -> a.T (3, 2);  a (2, 3) -> reshape(-1) (6,)
  #         E (400, 384) -> E.T (384, 400)
```

① `reshape(2, 3)` ordnet die sechs Werte von `np.arange(6)` in zwei Zeilen zu drei Spalten

② `-1` lässt NumPy die fehlende Länge ausrechnen: 6 Einträge in 3 Zeilen ergeben 2 Spalten

```
[[0 1 2]
 [3 4 5]]
[[0 1]
 [2 3]
 [4 5]]
[[0 3]
 [1 4]
 [2 5]]
[0 1 2 3 4 5] (6,)
(384, 400)
```

Überblick: Formen ändern

Ausdruck	Wirkung	Form
`E.T`	Zeilen und Spalten vertauscht	`(400, 384)` → `(384, 400)`
`q.T`	bei einer Achse ohne Wirkung	`(384,)` → `(384,)`
`a.reshape(3, 2)`	gleiche Einträge, neue Form, gleiche Anzahl	`(2, 3)` → `(3, 2)`
`a.reshape(-1)`	flach, `-1` rechnet die Länge aus	`(2, 3)` → `(6,)`
`q[None, :]`, `q[:, None]`	Achse der Länge 1 vorn bzw. hinten einfügen; `None` an einer Indexstelle steht für „hier eine neue Achse" (Kapitel 3.5)	`(384,)` → `(1, 384)` bzw. `(384, 1)`
`np.stack([E, E])`	gleich geformte Arrays entlang einer neuen ersten Achse stapeln	`(400, 384)` → `(2, 400, 384)`

`.T` und das Einfügen einer Achse ändern nur die Beschreibung des Arrays, nicht den Datenblock. `reshape` liefert eine Sicht, wenn sich die neue Form mit Abständen auf dem vorhandenen Block beschreiben lässt, sonst eine Kopie; `np.stack` legt immer ein neues Array an. Was das für den Speicher bedeutet, zeigt das Kapitel 3.7.

Vertiefung: Arrays zusammenfügen und aufteilen

Transponieren und Umformen verändern ein einzelnes Array. Oft müssen aber mehrere Arrays zu einem werden (die Embeddings aus zwei Dateien zu einer Matrix) oder eines in Teile zerfallen (Trainings- und Testdaten). Dafür gibt es `np.concatenate`, `np.stack` und `np.split`.

An `M` mit der Form `(3, 4)` und `a = np.arange(6).reshape(2, 3)`:

Aufruf	Wirkung	Ergebnisform
`np.concatenate([M, M], axis=0)`	Zeilen aneinanderhängen: die Achse 0 wird länger	`(6, 4)`
`np.concatenate([M, M], axis=1)`	Spalten aneinanderhängen: die Achse 1 wird länger	`(3, 8)`
`np.vstack([M, M])`, `np.hstack([M, M])`	Kurzformen für `axis=0` bzw. `axis=1`	`(6, 4)`, `(3, 8)`
`np.stack([M, M])`	eine neue Achse vorn, die Arrays liegen übereinander; alle müssen dieselbe Form haben	`(2, 3, 4)`
`np.split(M, 3, axis=0)`	in drei gleich große Teile entlang der Achse 0	drei Arrays `(1, 4)`
`np.split(M, [1], axis=0)`	an den angegebenen Positionen trennen: eine Zeile und der Rest	`(1, 4)` und `(2, 4)`
`np.array_split(E, 3)`	wie `split`, erlaubt aber ungleiche Teile (400 ist nicht durch 3 teilbar)	`(134, 384)`, `(133, 384)`, `(133, 384)`

`concatenate` verlangt gleiche Längen auf allen Achsen außer der genannten; `a` mit `(2, 3)` lässt sich nicht an `M` mit `(3, 4)` hängen. Alle Funktionen zum Zusammenfügen legen ein neues Array an. `split` liefert dagegen Sichten auf das Original.

3.3.6 Vektorisierung und Prüfzellen

Vektorisierung. Rechenzeichen und mathematische Funktionen wirken auf ein Array elementweise, ohne dass Sie eine Schleife schreiben: `E * E` quadriert alle 153.600 Einträge, `np.sqrt` zieht aus jedem die Wurzel, `E + 1.0` addiert zu jedem Eintrag 1. Diese Schreibweise heißt vektorisiert. Auch NumPy durchläuft dabei alle Einträge nacheinander in einer Schleife. Diese Schleife steht aber nicht in Ihrem Python-Code, sondern in der Bibliothek selbst, in kompiliertem C-Code, der über den zusammenhängenden Block läuft: ohne Interpreter, ohne Typprüfung je Eintrag und ohne Zahlenobjekte. Genau das ist der Unterschied, den die Messung am Anfang gezeigt hat. Funktionen aus der Standardbibliothek von Python wie `math.sqrt` erwarten dagegen eine einzelne Zahl; auf ein Array angewendet brechen sie mit `TypeError` ab, deshalb `np.sqrt`. Dieselbe Eigenschaft, eine Operation unabhängig auf jeden Eintrag anzuwenden, nutzt später die GPU: Dort verarbeiten tausende Rechenkerne die Einträge gleichzeitig statt nacheinander.

Vertiefung: Universelle Funktionen (ufuncs)

NICHT PRÜFUNGSRELEVANT

Viele elementweise Operationen von NumPy sind als universelle Funktionen implementiert, kurz ufuncs. Auch die Rechenzeichen gehören dazu: `E * E` ruft `np.multiply(E, E)` auf, `E + 1.0` ruft `np.add` auf. Eine ufunc bringt für die unterstützten numerischen Datentypen vorkompilierte Rechenroutinen mit und bringt Arrays verschiedener Form nach festen Regeln zusammen (Broadcasting, Kapitel 3.5).

Gruppe	Beispiele
Arithmetik	`np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`, `np.negative` (die Rechenzeichen `+ - * / ** -`)
Mathematische Funktionen	`np.sqrt`, `np.exp`, `np.log`, `np.abs`, `np.sin`, `np.cos`, `np.rint` (runden auf ganze Zahlen)
Vergleiche	`np.greater_equal` (`>=`), `np.equal` (`==`), `np.isfinite`, `np.isnan`
Zwei Arrays elementweise vergleichen	`np.maximum(a, b)`, `np.minimum(a, b)`: je Position der größere bzw. kleinere Wert, nicht zu verwechseln mit der Reduktion `a.max()`
Logik auf Masken	`np.logical_and` (`&`), `np.logical_or` (der senkrechte Strich), `np.logical_not` (`~`)

Nicht jede elementweise Funktion ist eine ufunc: `np.round` und `np.isclose` sind gewöhnliche Funktionen, die intern ufuncs aufrufen; `isinstance(np.isclose, np.ufunc)` ergibt `False`. Zwei Eigenschaften der ufuncs sind für große Daten interessant. Mit `out=` schreiben sie das Ergebnis in ein vorhandenes Array statt ein neues anzulegen: `np.multiply(x, x, out=squares)` spart die erneute Speicheranforderung, die Werte stehen aber weiterhin in `squares` und müssen von der nächsten Operation wieder gelesen werden. Und geeignete zweistellige ufuncs wie `np.add` und `np.maximum` lassen sich mit `reduce` und `accumulate` verwenden: `np.add.reduce(M, axis=1)` ist `M.sum(axis=1)`, `np.maximum.accumulate(s)` das laufende Maximum. Viele dieser Funktionen heißen in PyTorch gleich (`torch.sqrt`, `torch.maximum`), einzelne anders (`torch.round` statt `np.rint`, `torch.pow` statt `np.power`); das Wort ufunc gibt es dort nicht.

```
① squares = E * E
  print(squares.shape, squares.dtype)
② print(np.sqrt(np.array([9.0, 16.0, 25.0])))
  print((E + 1.0).dtype)                  # ein Python-float lässt ein float32-Array bei float32
  # Formen: E (400, 384) -> E * E (400, 384);  E (400, 384) -> E + 1.0 (400, 384)
```

① `*` zwischen zwei Arrays rechnet elementweise; die Schleife über alle Einträge läuft in NumPy, nicht in Python

② `np.sqrt` wirkt ebenfalls auf jeden Eintrag; `math.sqrt` würde mit TypeError abbrechen

```
(400, 384) float32
[3. 4. 5.]
float32
```

Vektorisierung verlagert die Schleife aus Python heraus, aber jedes Zwischenergebnis ist ein eigenes Array im Speicher. Eine Formel wie `np.sqrt((x * x).sum(axis=1, keepdims=True) + eps)` ist nicht eine Schleife, sondern eine Kette von Operationen, und jede legt ein neues Array an, das die nächste wieder einliest.

An der Formel des Operators lässt sich das abzählen. Für `x` mit der Form `(N, D)` entstehen nacheinander: `x * x` mit `(N, D)`, so groß wie die Eingabe; die Summe mit `(N, 1)`; die Summe plus `eps` mit `(N, 1)`; die Wurzel mit `(N, 1)`; und schließlich `x / norms` mit `(N, D)`, wieder so groß wie die Eingabe. Fünf neue Arrays, zwei davon in voller Größe. Für die Kursdaten mit 614.400 Bytes ist das unerheblich. Bei Arrays, die einen großen Teil des Speichers belegen, wird das Schreiben und Wiederlesen dieser Zwischenergebnisse zum Engpass, weil der Weg zum Speicher langsamer ist als die Rechnung selbst.

Später im Semester sehen Sie, wie eigene GPU-Programme (Kernel) mehrere Schritte zu einem Durchlauf zusammenfassen und diese Zwischenergebnisse gar nicht erst anlegen; in PyTorch übernimmt das der Compiler `torch.compile`. Davon zu unterscheiden sind In-place-Operationen wie `x *= x` statt `y = x * x`: Sie sparen den Speicher für ein weiteres Array, weil sie das vorhandene überschreiben, lesen und schreiben die Werte aber weiterhin. Auf der Eingabe des Operators sind sie ausgeschlossen, weil der Vertrag die Eingabe unverändert lässt; auf eigenen Zwischenergebnissen wie `norms` oder auf einer Kopie der Eingabe wären sie erlaubt.

✏️ Aufgabe 3.3.6 #1 — 5 min

Schreiben Sie die Funktion `row_norms(x)`. Sie bekommt ein zweidimensionales Array und gibt für jede Zeile die geometrische Länge des Vektors zurück, der in ihr steht (also die Wurzel aus der Summe der Quadrate seiner Einträge, nicht die Anzahl der Einträge). Das Ergebnis hat eine Achse mit einem Eintrag je Zeile. Verwenden Sie `*`, `sum` mit `axis=` und `np.sqrt`. Die Prüfzelle vergleicht mit `np.linalg.norm`, der Längenfunktion aus dem Unterpaket für lineare Algebra (Vertiefung in Kapitel 3.3.1), die der Appetizer der ersten Lerneinheit verwendet hat. Die Prüfzelle vergleicht mit einer Toleranz; was sie im Einzelnen prüft, steht nach der Aufgabe.

Wer früher fertig ist: Berechnen Sie in einer neuen Zelle `row_norms(E).min()` und `row_norms(E).max()`. Die letzte Lerneinheit hat dieselben Werte mit einer Schleife bestimmt (kürzester und längster Absatzvektor). Stimmen sie überein?

Hinweis: Die Summe soll über die Spalten laufen, also über die zweite Achse.

```
def row_norms(x):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
x = np.array([[3.0, 4.0], [0.0, 2.0]], dtype=np.float32)     # zwei Zeilen verschiedener Länge
result = row_norms(x)
assert result is not None, "row_norms gibt noch nichts zurück (return vergessen?)"
① assert result.shape == (2,), f"ein Wert je Zeile, Form (2,) statt {result.shape}"
assert result.dtype == np.float32, "der Datentyp der Eingabe bleibt erhalten"
② assert np.allclose(result, [5.0, 2.0]), "Zeile [3, 4] hat die Länge 5, Zeile [0, 2] die Länge 2"
assert np.allclose(row_norms(E), np.linalg.norm(E, axis=1)), "gleiche Werte wie np.linalg.norm über die Zeilen"
assert row_norms(np.zeros((0, 3), dtype=np.float32)).shape == (0,), "ein Array ohne Zeilen ergibt ein leeres Ergebnis"
print("Alle Prüfungen bestanden ✓")
```

① zuerst die Form prüfen: `(2,)` ist ein Tupel mit einem Eintrag, die Form eines Arrays mit einer Achse und zwei Einträgen
② `np.allclose` vergleicht alle Werte mit einer Toleranz

Was die Prüfzelle geprüft hat. Die Prüfzellen dieses Notebooks prüfen bei Arrays drei Dinge nacheinander: die Form, den Datentyp und erst dann die Werte. Bei den Werten prüfen sie, ob jeder Eintrag des Ergebnisses bis auf eine kleine Toleranz mit dem erwarteten Wert übereinstimmt. Dafür gibt es `np.allclose(a, b)`: Es vergleicht zwei Arrays Eintrag für Eintrag und liefert `True`, wenn alle Paare innerhalb der Toleranz liegen, so wie `abs(a - b) < 1e-9` bei einzelnen Zahlen. Form und Datentyp werden vorher getrennt geprüft, aus zwei Gründen. Erstens vergleicht `np.allclose` ein Array der Form `(1,`

`2)` ohne Fehlermeldung mit einem der Form `(2,)`, indem es die Werte wiederholt (die Regeln dafür heißen Broadcasting und kommen im Kapitel 3.5); ein Ergebnis mit falscher Form könnte so als richtig gelten. Zweitens stimmen die Werte eines `float64`-Ergebnisses mit denen eines `float32`-Ergebnisses überein, der Speicherbedarf aber nicht.

Warum `if arr:` nicht funktioniert. `if arr:` verwendet ein Array direkt als Bedingung, so wie `if liste:` in Grundkurs Teil 1 eine leere Liste von einer gefüllten unterschieden hat. Bei einem Array mit mehreren Einträgen geht das nicht: NumPy bricht mit `ValueError: The truth value of an array with more than one element is ambiguous` ab. Der Grund steckt im Wort „ambiguous": Hinter einem solchen `if` können drei verschiedene Fragen stehen, und NumPy rät nicht, welche gemeint ist. Jede Frage hat ihre eigene Schreibweise:

Frage	Schreibweise
Ist mindestens ein Eintrag wahr?	`np.any(arr)`
Sind alle Einträge wahr?	`np.all(arr)`
Hat das Array überhaupt Einträge?	`arr.size > 0`

Nur ein Array mit genau einem Eintrag lässt sich direkt als Wahrheitswert verwenden. Dieselbe Regel gilt für `and` und `or` zwischen zwei Arrays; dort stehen `&` und `|`, wie das Kapitel 3.6 zeigt.

```
flags = np.array([False, True, False])
print(np.any(flags), np.all(flags), flags.size > 0)
# if flags: ...   würde mit ValueError abbrechen, weil flags drei Einträge hat
```

```
True False True
```

3.4 Der Python-Data-Science-Werkzeugkasten

Welche Bibliothek wofür?

Das Array von NumPy ist die gemeinsame Sprache der Python-Bibliotheken für Daten und KI: Alle können NumPy-Arrays verarbeiten und ineinander überführen. pandas ergänzt eigene beschriftete Tabellenstrukturen (`DataFrame`, `Series`), die sich aus Arrays bauen und als Arrays auslesen lassen; scikit-learn nimmt Arrays als Trainingsdaten entgegen; Matplotlib zeichnet sie; PyTorch hat eigene Tensorobjekte, die sich ohne Umweg aus NumPy-Arrays erzeugen und dorthin zurückholen lassen. Mit NumPy im Rücken finden Sie sich in allen diesen Bibliotheken zurecht. Deshalb steht NumPy in diesem Kurs am Anfang, und deshalb spricht man vom Python-Data-Science-Werkzeugkasten: ein Satz Bibliotheken, die zusammenpassen, weil ihre Schnittstellen auf dem Array aufbauen.

Bibliothek	Wofür sie gut ist	Im Kurs
NumPy	Arrays und Rechnen auf ihnen, Grundlage der anderen	in dieser Lerneinheit
pandas	Tabellen mit benannten Spalten: Daten laden, bereinigen, zusammenfassen	nächste Lerneinheit
Matplotlib	Diagramme und Darstellungen aus Arrays	nächste Lerneinheit, in dieser einmal kurz
scikit-learn	klassisches maschinelles Lernen mit einer einheitlichen Schnittstelle für alle Verfahren	im Lauf des Semesters
PyTorch	Tensoren mit GPU-Unterstützung und automatischer Ableitung, Training neuronaler Netze	der Hauptteil des Semesters

Daneben werden Ihnen in Dokumentationen, Blogbeiträgen und Stellenanzeigen weitere Namen begegnen. Für jede Aufgabe des Kurses genügt eine der fünf Bibliotheken oben, deshalb behandeln wir diese nicht; einordnen können sollten Sie sie trotzdem:

Bibliothek	Einordnung
SciPy	numerische Verfahren auf NumPy-Arrays: Optimierung, Statistik, Signalverarbeitung
JAX, **TensorFlow**	Alternativen zu PyTorch für Tensoren und neuronale Netze
CuPy	die NumPy-Schnittstelle auf der GPU
Numba	übersetzt Python-Schleifen in Maschinencode
Keras	vereinfachte Schnittstelle zum Bau neuronaler Netze, setzt auf TensorFlow, JAX oder PyTorch auf
Polars	eine schnellere Alternative zu pandas für große Tabellen

Die Liste ist nicht vollständig; für Bilder (Pillow, OpenCV), Text (spaCy, Hugging Face Transformers) und große Datenmengen (Dask) gibt es weitere Bibliotheken, die alle NumPy-Arrays annehmen oder liefern.

3.5 Broadcasting und unser Semester-Operator

Mit `row_norms` kennen Sie die Länge des Vektors in jeder Zeile. Um jede Zeile zu normalisieren, muss jede Zeile durch ihre eigene Länge geteilt werden: eine Rechnung zwischen einem Array der Form `(400, 384)` und einem der Form `(400,)`. Der naheliegende Ausdruck `E / row_norms(E)` scheitert, NumPy meldet `ValueError: operands could not be broadcast together with shapes (400,384) (400,)`. Rechnungen zwischen Arrays verschiedener Form kommen beim Rechnen mit Arrays ständig vor, zum Beispiel eine Konstante zu jedem Eintrag addieren, von jeder Zeile den Mittelwert ihrer Spalte abziehen, jede Zeile durch ihren eigenen Wert teilen. NumPy regelt sie alle nach denselben wenigen Regeln. Sie heißen Broadcasting, und mit ihnen versteht man sowohl die Fehlermeldung als auch den Ausdruck, der stattdessen funktioniert.

Broadcasting: Formen werden von rechts verglichen. NumPy legt die beiden Formen rechtsbündig untereinander und vergleicht Achse für Achse. Zwei Achsen passen zusammen, wenn sie gleich viele Einträge haben oder eine von ihnen die Länge 1 hat; die 1 wird dann gedehnt. Gedehnt heißt: Der eine Eintrag dieser Achse wird für jeden Eintrag der längeren Achse wiederverwendet, so als stünde er dort mehrfach, ohne dass NumPy die Daten kopiert. Fehlt einer Form links eine Achse, wird sie als 1 ergänzt.

`(400, 384)` gegen `(400,)`: Rechts stehen 384 und 400, weder gleich noch eine 1, also ein Fehler. `(400, 384)` gegen `(400, 1)`: Rechts stehen 384 und 1, die 1 wird gedehnt, links stehen 400 und 400. Das passt, und jede Zeile bekommt ihren eigenen Teiler. Die Form `(400, 1)` entsteht aus `(400,)` mit `[:, None]` oder gleich bei der Reduktion mit `keepdims=True`.

Broadcasting: Formen werden von rechts verglichen

Zwei Achsen passen, wenn sie gleich viele Einträge haben oder eine die Länge 1 hat. Fehlende Achsen werden links als 1 ergänzt.

`E / norms[:, None]`

```
(400, 384)
(400,   1)
----------
(400, 384)
```

rechts: 384 gegen 1, die 1 wird auf 384 gedehnt
links: 400 gegen 400, gleich
je Zeile ein eigener Teiler

`E + q`

```
(400, 384)
(     384,)
----------
(400, 384)
```

q hat nur eine Achse
links wird eine 1 ergänzt: (1, 384)
q wird auf jede Zeile gelegt

`E / norms`

```
(400, 384)
(     400,)
----------
Fehler
```

rechts: 384 gegen 400
weder gleich noch eine 1
ValueError: could not be broadcast

So sieht es in den Werten aus, an x mit shape (2, 2) und den Zeilenlängen 5 und 2:

richtig

x (2, 2)		/	(2, 1)		→	gedehnt (2, 2)		=	Ergebnis	
3	4		5			5	5		0.6	0.8
0	2		2			2	2		0	1

x / norms[:, None], die Längen als Spalte (2, 1): die Spalte wird nach rechts gedehnt, der Wert 5 gilt für die ganze erste Zeile. Jede Zeile wird durch ihre eigene Länge geteilt.

stille Falle

x (2, 2)		/	(2,)		→	gedehnt (2, 2)		=	Ergebnis	
3	4		5	2		5	2		0.6	2.0
0	2					5	2		0	1

x / norms mit den Längen flach (2,): läuft ohne Fehler, ist aber falsch. (2,) wird links zu (1, 2) ergänzt und nach unten gedehnt: Die Spalten werden durch 5 und 2 geteilt, nicht die Zeilen.

Gedehnt heißt: Der eine Eintrag der Achse wird für jeden Eintrag der längeren Achse wiederverwendet, ohne dass NumPy die Daten kopiert.



Broadcasting legt ein kleineres Array in einem Zug auf jede Zeile des größeren, so wie der breite Stempel alle Karten gleichzeitig trifft. Ob der Stempel passt, entscheiden die Formen.

```
   norms = row_norms(E)
   print(norms.shape, "<- passt nicht gegen (400, 384)")
① print(norms[:, None].shape, "<- passt: die 1 wird auf 384 gedehnt")
   En = E / norms[:, None]
   print(En.shape, En.dtype)
   print("Länge der ersten drei Zeilen nach dem Teilen:", np.round(row_norms(En)[:3], 4))
   # Formen: E (400, 384) -> norms (400,) -> norms[:, None] (400, 1) -> E / norms[:, None] (400, 384)
```

① `None` an einer Indexposition fügt dort eine neue Achse der Länge 1 ein (NumPy nennt das auch `np.newaxis`): aus `(400,)` wird mit `[:, None]` die Form `(400, 1)`

```
(400,) <- passt nicht gegen (400, 384)
(400, 1) <- passt: die 1 wird auf 384 gedehnt
(400, 384) float32
Länge der ersten drei Zeilen nach dem Teilen: [1. 1. 1.]
```

✏️ Aufgabe 3.5 #1

2 min

Die stille Falle. Die Zelle darunter soll die zwei Zeilen von `x` normalisieren. Sie läuft ohne Fehler und liefert trotzdem ein falsches Ergebnis. Führen Sie die Zelle aus, erklären Sie in einem Kommentar, durch welche Werte NumPy hier tatsächlich geteilt hat, und korrigieren Sie die Zeile. Die Prüfzelle kennt das richtige Ergebnis.

Hinweis: `x` hat die Form `(2, 2)`, `norms` die Form `(2,)`. Legen Sie beide Formen rechtsbündig untereinander. Sie passen, und genau das ist das Problem: `norms` wird auf jede Zeile gelegt, nicht neben jede Zeile.

```
   x = np.array([[3.0, 4.0], [0.0, 2.0]], dtype=np.float32)
   norms = row_norms(x)                    # [5, 2]
   y = x / norms
   print(y)
```

```
   # Prüfzelle – einfach ausführen
   assert y.shape == (2, 2)
   assert np.allclose(y, [[0.6, 0.8], [0.0, 1.0]]), "jede Zeile durch ihre eigene Länge: [3, 4] / 5 und [0, 2]
/ 2"
   print("Alle Prüfungen bestanden ✓")
```

Überblick: Broadcasting

Formen werden rechtsbündig untereinandergelegt und von rechts verglichen. Zwei Achsen passen, wenn sie gleich viele Einträge haben oder eine die Länge 1 hat; die 1 wird gedehnt. Fehlende Achsen werden links als 1 ergänzt.

Ausdruck	Formen	Ergebnis
`E + 1.0`	`(400, 384)` gegen `()`	`(400, 384)`: die Zahl wird zu jedem Eintrag addiert
`E + q`	`(400, 384)` gegen `(384,)`	`(400, 384)`: `q` wird zu jeder Zeile addiert, Eintrag für Eintrag
`E / norms[:, None]`	`(400, 384)` gegen `(400, 1)`	`(400, 384)`: jede Zeile wird durch ihren eigenen Wert geteilt
`E / norms`	`(400, 384)` gegen `(400,)`	`ValueError`: 384 gegen 400 passt nicht
`Q[:, None, :] - E[None, :, :]`	`(8, 1, 384)` gegen `(1, 400, 384)`	`(8, 400, 384)`: für jede der 8 Anfragen und jeden der 400 Absätze die 384 Differenzen der Komponenten, also jede Anfrage minus jeder Absatz

Die stille Falle: `x / norms` mit `x (2, 2)` und `norms (2,)` läuft ohne Fehler und teilt spaltenweise statt zeilenweise. Deshalb die Achse ausdrücklich behalten: `keepdims=True` oder `[:, None]`.

Broadcasting vervielfacht die Eingabedaten nicht im Speicher, das Ergebnis braucht aber Speicher in seiner Ergebnisform: Das Paarbeispiel `(8, 400, 384)` sind in `float32` rund 4,9 MB.

💭 Kurz überlegen

Welche Form hat das Ergebnis der drei Ausdrücke `E + Q[0]`, `Q[:, None, :] - E[None, :, :]` und `E * E.sum(axis=1, keepdims=True)`? Schreiben Sie die Formen auf, oder stellen Sie fest, dass ein Ausdruck scheitert.

3.5.1 Der Semester-Operator row_l2_normalize

🔄 Wiederholung: normalize_eps

In LE 2, Abschnitt 2.6 bekam die Normalisierung einen kleinen positiven Wert `eps` unter der Wurzel, damit der Nullvektor keine Division durch null auslöst. Der Nullvektor bleibt dabei ein Nullvektor, weil der Zähler null ist. Der Preis: Die Länge des Ergebnisses ist $\|x\| \,/\, \sqrt{\|x\|^2 + \text{eps}}$ und nicht genau 1. Für die Absatzvektoren mit Längen zwischen 1,8 und 4,1 ist die Abweichung winzig, für sehr kurze Vektoren kann sie groß werden. `[0.0001, 0]` hat nach der Rechnung nur die Länge 0,0995. Das ist eine stabilisierte Normalisierung, keine exakte.

Jetzt bekommt diese Rechnung ihre endgültige Form als Operator auf einer ganzen Matrix. `row_l2_normalize` normalisiert jede Zeile eines Arrays. Es ist der Operator des Semesters: Sie schreiben ihn in dieser Lerneinheit in NumPy, später in PyTorch, als CUDA-Kernel und in Mojo. Damit alle Fassungen gegen dieselben Tests laufen können, bekommt er einen Vertrag.

Definition: Operator-Vertrag

Ein Operator-Vertrag legt für eine Funktion schriftlich fest, welche Eingaben sie annimmt, was sie zurückgibt, wie sie sich an den Randfällen verhält und wie sie ungültige Eingaben abweist. Alle Fassungen eines Operators, in welcher Sprache oder Bibliothek auch immer, werden gegen denselben Vertrag getestet; ohne ihn ließe sich nicht entscheiden, ob verschiedene Fassungen „dasselbe" tun.

Genau verstehen: Der Vertrag von row_l2_normalize

`row_l2_normalize(x, eps=1e-6)` normalisiert jede Zeile eines zweidimensionalen Arrays mit der stabilisierten Formel

$$\hat{x}_i = \frac{x_i}{\sqrt{\sum_j x_{ij}^2 + \varepsilon}}$$

für jede Zeile x_i mit D Einträgen. Das `eps` hält den Nenner größer als null. Für eine Nullzeile ist der Zähler null und der Nenner `sqrt(eps)`, die Zeile bleibt null. Die fünf Punkte des Vertrags:

1. **Eingabe:** ein zweidimensionales Array der Form `[N, D]` mit `N ≥ 0` und `D > 0`, Datentyp `float32`, zeilenweise zusammenhängend im Speicher.
2. **Ausgabe:** ein neues Array mit derselben Form und demselben Datentyp.
3. **Die Eingabe bleibt unverändert.**
4. **Nullzeilen bleiben Nullzeilen.**
5. **`eps`** ist positiv und endlich, und zwar im Rechentyp `float32`.

Ungültige Eingaben weist der Operator mit einem `ValueError` ab: falsche Anzahl der Achsen, falscher Datentyp, `D = 0`, ungültiges `eps`, nicht zeilenweise zusammenhängend. Was „zeilenweise zusammenhängend" bedeutet, erklärt das Kapitel 3.7; dort wird die Prüfung ergänzt und der Operator damit vervollständigt. Die Einschränkung braucht NumPy nicht, sie ist ein Vertrag für die späteren Implementierungen, die ein dichtes zeilenweises Speicherlayout voraussetzen.

Eine Formel, viele Stacks. Genau diese Formel implementieren Sie im Semester mehrmals. Was sich ändert, ist nie die Mathematik, sondern wo die Schleife läuft und wer sie schreibt.

In NumPy schreibt die Bibliothek die Schleife, vektorisiert und in kompiliertem Code. In PyTorch wählt eine Verteilstelle (der Dispatcher) anhand der Eigenschaften der Tensoren die passende Implementierung; das Gerät legen Sie selbst fest, indem Sie die Tensoren dort anlegen oder dorthin verschieben. In CUDA schreiben Sie die Schleife selbst als Programm für die GPU (einen Kernel), in dem viele Threads gleichzeitig je einen Teil der Arbeit übernehmen, und in Mojo beschreiben Sie dasselbe in einer höheren Sprache, die trotzdem zu einem solchen Programm übersetzt wird. Der Vertrag ist das, was alle Fassungen verbindet: Jede muss dieselben Randfälle bestehen.

Die Eingabeprüfung als Vorgabe. Die Prüfungen des Vertrags stehen in der Funktion `check_input`, die Ihre Fassung des Operators als erste Zeile aufruft. Sie verwendet `raise ValueError(...)`, womit eine Funktion einen Fehler auslöst und abbricht. Fehler auslösen und behandeln ist Thema von Grundkurs Teil 3, deshalb ist diese Zelle eine Vorgabe. Lesen Sie sie trotzdem: Jede Zeile entspricht einem Punkt des Vertrags.

```python
# Vorgabe: die Eingabeprüfung des Operators — einfach ausführen
def check_input(x, eps):
    if x.ndim != 2:                                                    # ①
        raise ValueError(f"Eingabe muss zwei Achsen haben, hat {x.ndim}")
    if x.dtype != np.float32:
        raise ValueError(f"Eingabe muss float32 sein, ist {x.dtype}")
    if x.shape[1] == 0:
        raise ValueError("Eingabe muss mindestens eine Spalte haben (D > 0)")
    eps32 = np.float32(eps)                                            # ②
    if not np.isfinite(eps32) or eps32 <= 0:
        raise ValueError(f"eps muss in float32 positiv und endlich sein, ist {eps}")
    # Die Prüfung auf zeilenweise zusammenhängenden Speicher kommt im Kapitel „Sicht oder Kopie" dazu.
```

① `raise ValueError("...")` bricht die Funktion mit dieser Fehlermeldung ab
② `np.float32(eps)` rechnet eps in den Rechentyp um; `np.isfinite` ist False für unendlich und NaN

`1e-100` ist als Python-Zahl positiv, in `float32` aber null: Der kleinste darstellbare positive Wert dieses Typs liegt bei etwa `1e-45`. Mit einem solchen `eps` würde eine Nullzeile zu `NaN`, der Darstellung von „keine Zahl" nach einer Division 0 durch 0. Deshalb prüft `check_input` den Wert im Rechentyp `float32`. Ein `eps` schützt nur den Nenner vor null. Vor einem Überlauf beim Quadrieren sehr großer Werte schützt es nicht.

✏️ Aufgabe 3.5.1 #1 10 min

Schreiben Sie die Funktion `row_l2_normalize(x, eps=1e-6)` nach dem Vertrag. Die erste Zeile ruft `check_input(x, eps)` auf, die zweite wandelt `eps` mit `eps = np.float32(eps)` in den Rechentyp um. Danach berechnen Sie für jede Zeile die Summe der Quadrate, addieren `eps`, ziehen die Wurzel und teilen die Zeile durch das Ergebnis. Achten Sie auf die Form des Nenners. Das Ergebnis muss `float32` bleiben und die Eingabe unverändert lassen.

Wer früher fertig ist: Schreiben Sie in einer neuen Zelle einen eigenen Test mit dem Array `[[3, 4], [0, 0], [1, 1]]` als `float32`. Prüfen Sie mit `row_norms`, dass die erste und die dritte Zeile nach dem Operator näherungsweise die Länge 1 haben (`np.allclose` mit `atol=1e-6`) und die Nullzeile die Länge 0.

Hinweis: `(x * x).sum(axis=1, keepdims=True)` liefert die Summen der Quadrate schon in der Form `(N, 1)`, die gegen `(N, D)` passt. Die Umwandlung von `eps` sorgt dafür, dass auch ein `eps` vom Typ `float64` das Ergebnis nicht auf `float64` anhebt.

```python
def row_l2_normalize(x, eps=1e-6):
    # TODO: Ihre Lösung hier
    ...
```

```python
# Prüfzelle – einfach ausführen
x = np.array([[3.0, 4.0], [0.0, 2.0]], dtype=np.float32)     # zwei Zeilen verschiedener Länge
x_before = x.copy()                                           # Kopie für den Vergleich danach
y = row_l2_normalize(x)
assert y is not None, "row_l2_normalize gibt noch nichts zurück (return vergessen?)"
assert y.shape == (2, 2), f"gleiche Form wie die Eingabe, {y.shape} statt (2, 2)"
assert y.dtype == np.float32, f"gleicher Datentyp wie die Eingabe, {y.dtype} statt float32"
assert np.allclose(y, [[0.6, 0.8], [0.0, 1.0]], atol=1e-6), "jede Zeile durch ihre eigene Länge"
assert np.array_equal(x, x_before), "die Eingabe muss unverändert bleiben"
assert np.all(row_l2_normalize(np.zeros((1, 3), dtype=np.float32)) == 0), "eine Nullzeile bleibt eine Nullzeile"
tiny = row_l2_normalize(np.array([[1e-4, 0.0]], dtype=np.float32))
assert abs(row_norms(tiny)[0] - 0.0995) < 1e-3, "eine sehr kurze Zeile wird nicht auf Länge 1 gebracht (stabilisierte Formel)"
assert row_l2_normalize(np.zeros((0, 3), dtype=np.float32)).shape == (0, 3), "ein Array ohne Zeilen ist zulässig"
assert row_l2_normalize(x, eps=np.float64(1e-6)).dtype == np.float32, "auch ein eps vom Typ float64 darf das Ergebnis nicht anheben (eps = np.float32(eps))"
assert np.allclose(row_l2_normalize(x, eps=1.0)[0], [3 / 26 ** 0.5, 4 / 26 ** 0.5], atol=1e-6), "eps muss in die Rechnung eingehen: [3, 4] mit eps=1 ergibt den Nenner sqrt(26)"
try:
    row_l2_normalize(np.zeros((2, 2)))                        # float64 statt float32
    raise AssertionError("float64 muss abgewiesen werden: ruft Ihre Funktion check_input auf?")
except ValueError:
    pass
try:
    row_l2_normalize(np.zeros(3, dtype=np.float32))           # eine Achse statt zwei
    raise AssertionError("ein Array mit einer Achse muss abgewiesen werden")
except ValueError:
    pass
print("Alle Prüfungen bestanden ✓")
```

① `.copy()` legt eine unabhängige Kopie des Arrays an, mehr dazu im Kapitel „Sicht oder Kopie"
② `np.array_equal` prüft, ob zwei Arrays dieselbe Form und exakt dieselben Werte haben, ohne Toleranz
③ `try` führt den Block aus, `except ValueError` fängt genau diesen Fehler ab, mehr dazu in Grundkurs Teil 3

✏️ Aufgabe 3.5.1 #2 2 min

Review. Die Fassung `row_l2_normalize_v2` in der Zelle darunter stammt aus einem KI-Assistenten. Sie besteht einen Test mit einer einzelnen Zeile. Lesen Sie den Code und nennen Sie in der zweiten Zelle als `assert` einen Testfall, bei dem diese Fassung scheitert und Ihre besteht. Führen Sie dann beide Zellen aus.

Hinweis: Was berechnet `np.linalg.norm(x)` ohne `axis` für ein Array mit zwei Zeilen?

```
def row_l2_normalize_v2(x, eps=1e-6):
    check_input(x, eps)
    return x / np.sqrt(np.linalg.norm(x) ** 2 + eps)

print(row_l2_normalize_v2(np.array([[3.0, 4.0]], dtype=np.float32)))     # eine Zeile: richtig
```

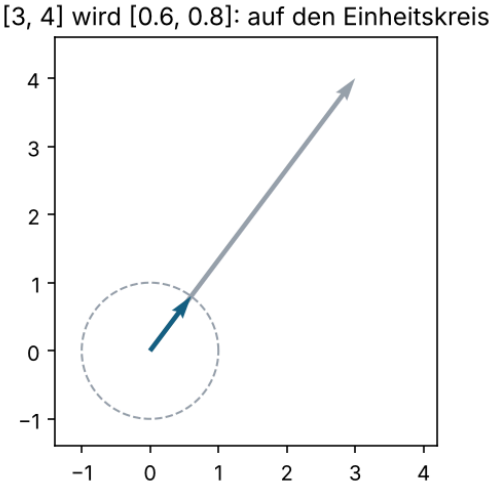
```
# Ihr Testfall als assert, dahinter als Kommentar, welchen Fehler er erkennt:
# TODO: Ihre Lösung hier
...
```

Ein Test mit nur einer Zeile unterscheidet nicht zwischen „jede Zeile durch ihre Länge" und „alle Zeilen durch dieselbe Länge". Deshalb enthalten die Prüfzellen dieses Notebooks mindestens zwei Zeilen mit verschiedener Länge. Bei quadratischen Arrays läuft die falsche Broadcasting-Rechnung aus der stillen Falle ohne Formfehler durch, ein Vergleich mit den erwarteten Werten erkennt sie trotzdem. Nichtquadratische Testdaten machen zusätzlich die Formverwechslung sichtbar, dann meldet schon die Form den Fehler.

Zum Schluss die Rechnung als Bild: Die Zeile `[3, 4]` hat die geometrische Länge 5 und landet nach dem Operator bei `[0.6, 0.8]`, auf dem Einheitskreis. Die Zelle verwendet Matplotlib, das in der nächsten Lerneinheit ausführlich drankommt.

```
import matplotlib.pyplot as plt

v = np.array([[3.0, 4.0]], dtype=np.float32)
u = row_l2_normalize(v)[0]
fig, ax = plt.subplots(figsize=(4.2, 3.6))
t = np.linspace(0, 2 * np.pi, 200)
ax.plot(np.cos(t), np.sin(t), "--", color="#98a1ad", lw=1)                # der Einheitskreis
ax.quiver([0, 0], [0, 0], [3, u[0]], [4, u[1]], angles="xy", scale_units="xy",
          scale=1, color=["#98a1ad", "#156082"], width=0.012)
ax.set_xlim(-1.4, 4.2); ax.set_ylim(-1.4, 4.6); ax.set_aspect("equal")
ax.set_title("[3, 4] wird [0.6, 0.8]: auf den Einheitskreis")
plt.show()
```



Warum der Datentyp kippen kann. Ein Python-`float` wie `1e-6` lässt ein `float32`-Array bei `float32`, `x * x + 1e-6` bleibt deshalb `float32`. Ein `float64`-Wert von NumPy, etwa `np.float64(1e-6)` oder ein `float64`-Array, hebt das Ergebnis dagegen auf `float64` an; für den NumPy-Skalar gilt das seit NumPy 2. (Ein Python-`float` auf einem Ganzzahlarray ergibt `float64`; hier geht es um Fließkomma-Arrays.) Ein solcher Wechsel verdoppelt den Speicher und fällt an den Werten nicht auf. Deshalb prüfen die Prüfzellen den Datentyp vor den Werten, und deshalb wandelt der Operator `eps` zuerst um. Typische Quellen für den stillen Wechsel sind `np.zeros` ohne `dtype`, Konstanten, die als Array angelegt wurden, und `float64`-Zwischenergebnisse aus anderen Bibliotheken.

Vertiefung: Wie groß darf eps sein? NICHT PRÜFUNGSRELEVANT

`eps` muss gegenüber den Quadratsummen der Zeilen klein sein, sonst verfälscht es alle Längen, und gegenüber `float32` groß genug, um nicht zu null zu werden. `1e-6` passt zu Vektoren mit Längen um 1 bis 10, wie unseren Absatzvektoren.

Unter der Wurzel addiert, wirkt `eps` wie ein zusätzlicher Eintrag der Zeile mit dem Quadrat `eps`. Bei einer Quadratsumme von 10 ändert `1e-6` die Länge rechnerisch um etwa `1.6e-7`, relativ um etwa `5e-8`; in `float32` ist diese Änderung so klein, dass sie beim Runden ganz verschwinden kann. Bei einer Quadratsumme von `1e-8` dominiert es. PyTorch macht es etwas anders: Es verwendet in seiner Normalisierungsfunktion das Maximum aus Länge und `eps` statt der Addition unter der Wurzel, mit `eps = 1e-12`; das Verhalten für Nullzeilen ist dasselbe.

3.5.2 Compare: alle Skalarprodukte auf einmal

Skalarprodukt, Matrixprodukt und elementweises Produkt

Das Skalarprodukt zweier Vektoren a und b mit gleich vielen Einträgen ist die Summe der Produkte ihrer Einträge; in NumPy schreibt man es `a @ b`:

$$a \cdot b = \sum_j a_j \, b_j$$

Für Einheitsvektoren ist es der Kosinus des Winkels zwischen beiden, deshalb misst es in der Suche die Ähnlichkeit von Anfrage und Absatz. Das Matrixprodukt einer Matrix A der Form (m, d) mit einer Matrix B der Form (d, n), in NumPy `A @ B`, ist die Matrix der Form (m, n) aller Skalarprodukte zwischen den Zeilen von A und den Spalten von B:

$$(AB)_{ik} = \sum_j A_{ij} \, B_{jk}$$

Es berechnet also viele Skalarprodukte auf einmal, genau das, was die Suche mit allen Absätzen braucht. Dieselbe Operation ist der Kern jeder vollverbundenen Schicht eines neuronalen Netzes. Große Matrixprodukte eignen sich besonders gut für die parallele Ausführung auf der GPU; auch elementweise Operationen und Reduktionen, aus denen unser Operator besteht, profitieren bei ausreichender Größe davon.

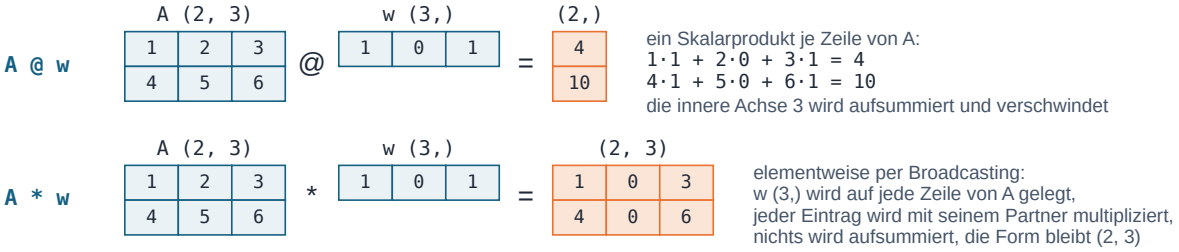
Nicht zu verwechseln damit ist das elementweise Produkt zweier gleich geformter Matrizen, in der Mathematik Hadamard-Produkt genannt und mit einem Kreis geschrieben, in NumPy das gewöhnliche `A * B`. Dabei wird nichts aufsummiert:

$$(A \odot B)_{ij} = A_{ij} \, B_{ij}$$

Das Ergebnis hat die Form der Eingaben, nicht (m, n). In der Mathematik müssen die Formen gleich sein, in NumPy gilt dafür Broadcasting. Die Grafik unten stellt `@` und `*` an einem Beispiel gegenüber.

Der zweite Schritt der Suche, Compare, berechnet das Skalarprodukt der Anfrage mit jedem der 400 Absätze. In NumPy ist das eine einzige Operation, das Matrixprodukt `@`: Es bildet für jede Zeile der linken Matrix und jede Spalte der rechten das Skalarprodukt, und mit der Anfrage als einziger Zeile links sind das genau die 400 gesuchten Werte. Vorher zwei Unterscheidungen an einer kleinen Matrix: `*` multipliziert elementweise, `@` bildet Summen von Produkten. Und `.T` liefert die passende Form für `@`, lässt ein eindimensionales Array aber unverändert.

A @ w und A * w: zwei verschiedene Rechnungen



Für die Suche brauchen wir @: Das Skalarprodukt der Anfrage mit jedem Absatz ist je Zeile eine Summe von Produkten.

```
  A = np.array([[1.0, 2.0, 3.0],
                [4.0, 5.0, 6.0]])            # Form (2, 3)
  w = np.array([1.0, 0.0, 1.0])              # Form (3,)
① print(A @ w)                               # [1*1 + 2*0 + 3*1, 4*1 + 5*0 + 6*1]
② print(A * w)
③ print(w.T.shape, w[:, None].shape, w[None, :].shape)
  # Formen: A (2, 3) @ w (3,) -> (2,);  A (2, 3) * w (3,) -> (2, 3)
  #         w.T (3,);  w[:, None] (3, 1);  w[None, :] (1, 3)
```

① `A @ w` bildet für jede Zeile von A die Summe der Produkte mit w: ein Skalarprodukt je Zeile
② `*` würde elementweise rechnen: (2, 3) mal (3,) ergibt per Broadcasting (2, 3), kein Skalarprodukt
③ `.T` lässt ein eindimensionales Array unverändert, erst eine zweite Achse macht daraus eine Spalte

```
[ 4. 10.]
[[1. 0. 3.]
 [4. 0. 6.]]
(3,) (3, 1) (1, 3)
```

Beim Matrixprodukt zweier zweidimensionaler Arrays muss die letzte Achse des linken zur vorletzten des rechten passen. `(8, 384) @ (384, 400)` ergibt `(8, 400)`: Die innere Achse 384 wird aufsummiert und verschwindet, die äußeren Achsen bilden das Ergebnis. Für Matrix mal Vektor gilt entsprechend `(m, d) @ (d,) -> (m,)`, das war das Beispiel `A @ w`. Diese Formenregel werden Sie im Semester ständig anwenden: Vollverbundene Schichten eines neuronalen Netzes multiplizieren ihre Eingabe mit einer Gewichtsmatrix, und auch bei allen anderen Schichttypen müssen Ein- und Ausgabeformen zusammenpassen.

Für eine einzelne Anfrage lautet der Formenweg `q (384,) -> q[None, :] (1, 384) -> (1, 384) @ (384, 400) -> (1, 400) -> [0] -> (400,)`. Die zusätzliche Achse am Anfang braucht `row_l2_normalize`, weil der Vertrag zwei Achsen verlangt: Aus der Anfrage wird eine Matrix mit einer Zeile. Das Matrixprodukt liefert dann eine Matrix mit einer Zeile und 400 Spalten, und `[0]` holt diese eine Zeile als Array der Form `(400,)` heraus.

✏️ Aufgabe 3.5.2 #1 5 min

Schreiben Sie die Funktion `cosine_scores(q, docs)`. Sie bekommt einen Anfragevektor `q` der Form `(D,)` und eine Dokumentmatrix `docs` der Form `(N, D)`, normalisiert beide mit `row_l2_normalize` und gibt die `N` Skalarprodukte als Array der Form `(N,)` zurück. Weil der Operator die stabilisierte Formel verwendet, sind das stabilisierte Näherungen der Kosinus-Ähnlichkeit: Für die Absatzvektoren fällt der Unterschied nicht ins Gewicht, für sehr kurze Vektoren schon. Die Prüfzelle vergleicht mit den Werten aus der letzten Lerneinheit und prüft genau diesen Unterschied.

Wer früher fertig ist: Geben Sie in einer neuen Zelle für Anfrage 0 die drei höchsten Werte aus `cosine_scores(Q[0], E)` aus. `np.sort` sortiert aufsteigend, die drei höchsten stehen also am Ende.

Hinweis: `q[None, :]` macht aus `(D,)` die Form `(1, D)`. Nach `(1, D) @ (D, N)` holt `[0]` die eine Zeile heraus. Die Form `(D, N)` entsteht aus der normalisierten Matrix mit `.T`.

```
def cosine_scores(q, docs):
    # TODO: Ihre Lösung hier
    ...
```

```
    # Prüfzelle – einfach ausführen
    scores = cosine_scores(Q[0], E)
    assert scores is not None, "cosine_scores gibt noch nichts zurück (return vergessen?)"
    assert scores.shape == (400,), f"ein Wert je Absatz, Form (400,) statt {scores.shape}"
    assert scores.dtype == np.float32, f"float32 wie die Eingabe, {scores.dtype} (geht ein float64-Wert in die Rechnung ein?)"
    assert cosine_scores(Q[0], E[:1]).shape == (1,), "auch bei genau einem Dokument bleibt es ein Array der Form (1,)"
① assert np.allclose(scores, scores_python(0), rtol=1e-5, atol=1e-6), "gleiche Werte wie die Suche von Hand (bis auf float32)"
    assert np.allclose(cosine_scores(Q[3], E), scores_python(3), rtol=1e-5, atol=1e-6)
    assert np.all(scores <= 1.0 + 1e-6) and np.all(scores >= -1.0 - 1e-6), "Kosinus-Werte liegen zwischen -1 und 1"
    tiny = np.array([1e-4, 0.0], dtype=np.float32)
    assert np.allclose(cosine_scores(tiny, tiny[None, :]), [0.00990099], rtol=1e-3), "sehr kurze Vektoren: die stabilisierte Formel liefert 0.0099, die exakte Kosinus-Ähnlichkeit wäre 1 (nutzt Ihre Fassung row_l2_normalize?)"
    assert np.all(cosine_scores(np.zeros(384, dtype=np.float32), E) == 0), "eine Nullanfrage ergibt überall 0"
    print("Alle Prüfungen bestanden ✓")
```

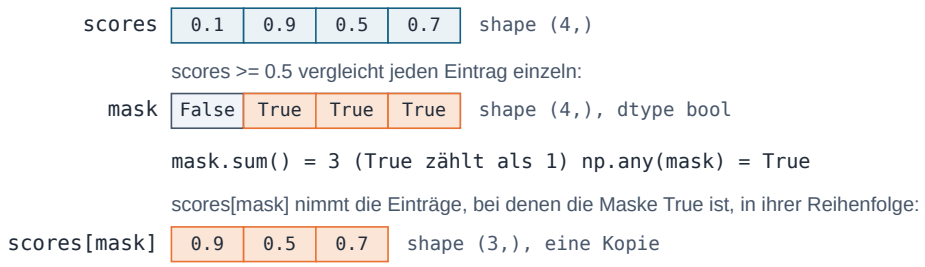
① die Werte der letzten Lerneinheit sind mit 64-Bit-Zahlen gerechnet, deshalb eine sichtbare Toleranz: rtol relativ, atol absolut

Die Toleranz in der Prüfzelle ist bewusst sichtbar: `rtol=1e-5` erlaubt eine relative Abweichung von einem Hunderttausendstel, `atol=1e-6` eine absolute von einem Millionstel. Die Werte der letzten Lerneinheit sind mit 64-Bit-Zahlen gerechnet, NumPy rechnet hier mit `float32` und rund sieben gültigen Stellen. Die Standardtoleranz von `allclose` ist enger und scheitert bei einzelnen Anfragen, obwohl die größte Abweichung unter einem Millionstel liegt.

3.6 Die Suche in NumPy

`cosine_scores(Q[0], E)` liefert 400 Werte. Der dritte Schritt, Select, wählt daraus aus: alle Absätze über einer Schwelle, oder die drei besten. Für die Auswahl nach einer Bedingung hat NumPy ein eigenes Werkzeug, das die bedingte Comprehension aus dem Aufwärmen ersetzt: die **Maske**. Ein Vergleich wie `scores >= 0.5` wirkt elementweise und liefert ein Array aus Wahrheitswerten in derselben Form. Eine Maske lässt sich zählen (wie viele Treffer?), prüfen (gibt es überhaupt einen?) und als Index verwenden (welche Werte sind es?). Masken sind das Standardwerkzeug für jede Auswahl nach Bedingung, auch in pandas und PyTorch.

Eine Maske: Vergleich, Zählen, Auswahl



Die Maske hat dieselbe Form wie scores. Als Index wirkt sie wie ein Sieb: Nur die Positionen mit True kommen durch.

```
   scores = cosine_scores(Q[0], E)
① mask = scores >= 0.5
   print(mask.shape, mask.dtype)
② print(mask.sum(), "Absätze mit Score ab 0.5,  mindestens einer:", np.any(mask))
③ print(scores[mask])
④ print(scores[(scores >= 0.3) & (scores < 0.5)].shape)
   # Formen: scores (400,) -> mask (400,) -> scores[mask] (Anzahl der True,)
```

① ein Vergleich mit einem Array liefert ein Array aus True und False, die Maske
② `sum` zählt die True-Werte, `np.any` sagt, ob mindestens einer dabei ist
③ eine Maske als Index wählt die Einträge aus, bei denen sie True ist
④ zwei Bedingungen verbindet `&`, jede in Klammern; `and` funktioniert hier nicht (dasselbe Problem wie bei `if arr`)

```
(400,) bool
1 Absätze mit Score ab 0.5,  mindestens einer: True
[0.83097094]
(10,)
```

✏️ Aufgabe 3.6 #1 2 min

Schreiben Sie die Funktion `between(scores, lo, hi)`. Sie gibt alle Scores mit `lo ≤ score ≤ hi` als eindimensionales Array in ihrer ursprünglichen Reihenfolge zurück. Vorausgesetzt wird `lo ≤ hi`. Die Prüfzelle enthält Werte genau auf beiden Grenzen.

```
   def between(scores, lo, hi):
       # TODO: Ihre Lösung hier
       ...
```

```
   # Prüfzelle — einfach ausführen
   s = np.array([0.125, 0.5, 0.875, 0.25, 0.75], dtype=np.float32)      # in float32 exakt darstellbar
   assert between(s, 0.25, 0.75) is not None, "between gibt noch nichts zurück (return vergessen?)"
   assert np.array_equal(between(s, 0.25, 0.75), [0.5, 0.25, 0.75]), "beide Grenzen gehören dazu, die
Reihenfolge bleibt"
   assert between(s, 0.95, 1.0).shape == (0,), "ohne Treffer ein leeres Array"
   assert between(scores, 0.5, 1.0).shape == (mask.sum(),)
   print("Alle Prüfungen bestanden ✓")
```

Select mit `argsort`. Für die Auswahl der besten Treffer braucht es nicht die Werte, sondern ihre Positionen, denn erst die Positionen führen zu den Titeln der Absätze. Dafür gibt es zwei Werkzeuge:

- Ein **Indexarray** ist ein Array aus Positionen. Es lässt sich wie eine Maske als Index verwenden: `scores[np.array([2, 0])]` liefert die Einträge 2 und 0 in dieser Reihenfolge.
- `np.argsort(scores)` liefert die Positionen, die `scores` aufsteigend sortieren würden, also ein Indexarray. Für die höchsten Werte zuerst sortiert man `-scores` aufsteigend. Bei gleichen Werten legt `kind="stable"` fest, dass die Reihenfolge der Eingabe erhalten bleibt.

Die Zeile aus dem Appetizer der ersten Lerneinheit, `np.argsort(scores)[::-1][:k]`, sortiert aufsteigend und dreht dann die Reihenfolge um. Das kehrt auch Gleichstände um, deshalb verwendet `top_k` die Fassung mit `-scores`.

```
   s = np.array([0.1, 0.9, 0.5], dtype=np.float32)
   print(s[np.array([2, 0])])
① print(np.argsort(s))
② print(np.argsort(-s, kind="stable"))
```

① `np.argsort` liefert Positionen, nicht Werte: s[1] ist der größte, s[0] der kleinste Eintrag
② `-s` dreht die Reihenfolge der Sortierung um, `kind="stable"` lässt gleiche Werte in der Eingabereihenfolge

```
[0.5 0.1]
[0 2 1]
[1 2 0]
```

✏️ Aufgabe 3.6 #2 5 min

Schreiben Sie die Funktion `top_k(scores, k)`. Sie gibt die Positionen der höchstens `k` höchsten Scores zurück, absteigend nach Score, bei Gleichstand in der Reihenfolge der Eingabe. `k` ist eine ganze Zahl ab 0. Für `k = 0` und für eine leere Eingabe ist das Ergebnis ein leeres Array, für `k` größer als die Anzahl der Scores alle Positionen.

Wer früher fertig ist: `np.argsort` nimmt auch `axis=`. Berechnen Sie in einer neuen Zelle für alle acht Anfragen auf einmal die Positionen der drei besten Absätze: Die Scores aller Anfragen liefert die Zelle im nächsten Abschnitt als Matrix der Form `(8, 400)`, `np.argsort(-S, axis=1, kind="stable")[:, :3]` hat dann die Form `(8, 3)`.

Hinweis: `np.argsort(-scores, kind="stable")` liefert alle Positionen in der richtigen Reihenfolge, ein Slice nimmt davon die ersten `k`. Ein Slice über das Ende hinaus ist bei Arrays wie bei Listen erlaubt.

```
def top_k(scores, k):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
s = np.array([0.1, 0.9, 0.5, 0.9, 0.3], dtype=np.float32)
assert top_k(s, 2) is not None, "top_k gibt noch nichts zurück (return vergessen?)"
① assert np.issubdtype(top_k(s, 2).dtype, np.integer), "Positionen müssen ganze Zahlen sein, sonst taugen sie nicht als Index"
assert np.array_equal(top_k(s, 2), [1, 3]), "zwei gleiche Höchstwerte: zuerst die frühere Position"
assert np.array_equal(top_k(s, 3), [1, 3, 2]), "absteigend nach Score"
assert top_k(s, 0).shape == (0,), "k = 0 ergibt ein leeres Array"
assert np.array_equal(top_k(s, 10), [1, 3, 2, 4, 0]), "k größer als die Anzahl: alle Positionen"
assert top_k(np.zeros(0, dtype=np.float32), 3).shape == (0,), "leere Eingabe"
s20 = np.array([0.9, 0.5] * 10, dtype=np.float32)      # 20 Einträge, zehn Gleichstände
assert np.array_equal(top_k(s20, 3), [0, 2, 4]), "Gleichstände in Eingabereihenfolge auch bei vielen Einträgen (kind='stable')"
assert top_k(scores, 1)[0] == 108, "Anfrage 0 hat Absatz 108 (San Diego) auf Platz 1"
print("Alle Prüfungen bestanden ✓")
```

① `np.issubdtype(dtype, np.integer)` prüft, ob ein Datentyp ein Ganzzahltyp ist; `np.array_equal` allein vergleicht nur die Werte, nicht den Typ

Überblick: Masken und Indexarrays

An `s = np.array([0.1, 0.9, 0.5, 0.7])`; für `scores` mit 400 Einträgen gilt dasselbe.

Ausdruck	Bedeutung	Ergebnis an `s`
`mask = s >= 0.5`	Maske: je Eintrag `True` oder `False`, Form wie `s`	`[False, True, True, True]`
`mask.sum()`	Anzahl der `True`-Einträge	`3`
`np.any(mask)`, `np.all(mask)`	ist mindestens ein Eintrag `True`? Sind alle Einträge `True`?	`True`, `False`
`s[mask]`	die Einträge, bei denen die Maske `True` ist, in der ursprünglichen Reihenfolge (Kopie)	`[0.9, 0.5, 0.7]`
`(s >= 0.3) & (s < 0.8)`	zwei Masken mit `&` (und) verknüpft, jede Bedingung in Klammern	`[False, False, True, True]`
`(s < 0.2) | (s > 0.8)`, `~mask`	oder-Verknüpfung mit dem senkrechten Strich, Verneinung mit `~`; `and`, `or`, `not` funktionieren bei Arrays nicht	`[True, True, False, False]`, `[True, False, False, False]`
`s[np.array([2, 0])]`	Indexarray: die Einträge an den Positionen 2 und 0, in dieser Reihenfolge (Kopie)	`[0.5, 0.1]`
`np.argsort(s)`	die Positionen, die `s` aufsteigend sortieren würden	`[0, 2, 3, 1]`
`i = np.argsort(-s, kind="stable")[:2]`	Positionen der zwei höchsten Werte, Gleichstände in Eingabereihenfolge	`[1, 3]`
`s[i]`	die Werte zu diesen Positionen, absteigend	`[0.9, 0.7]`

`argsort` liefert Positionen, keine Werte. Masken und Indexarrays liefern beim Lesen immer eine Kopie der ausgewählten Einträge, ein Slice dagegen eine Sicht (Kapitel 3.7).

Damit steht die Suche in NumPy. Die Ausgabe für Anfrage 0 können Sie mit der Fassung von Hand aus der letzten Lerneinheit vergleichen, Treffer und Werte stimmen überein.

```
scores = cosine_scores(Q[0], E)                # Auswahl und Ausgabe aus denselben Werten
print("Anfrage:", queries[0]["text"])
for i in top_k(scores, 3):
    print(f"  {scores[i]:.3f}  {passages[i]['title']}")
```

```
Anfrage: Wo liegt San Diego?
  0.831  San Diego
  0.351  Mexiko-Stadt
  0.338  North Carolina
```

Vertiefung: Geht es schneller als mit argsort? NICHT PRÜFUNGSRELEVANT

`np.argsort` sortiert alle 400 Werte, obwohl `top_k` nur die drei höchsten braucht. `np.argpartition` erledigt weniger Arbeit: `idx = np.argpartition(-scores, k - 1)[:k]` liefert für `0 < k <= len(scores)` die Positionen der `k` höchsten Werte, in beliebiger Reihenfolge. Bei 400 Werten spielt der Unterschied keine Rolle, bei Millionen schon.

`argpartition` gibt zunächst ein vollständiges Indexarray zurück, in dem die `k` gesuchten Positionen vorn stehen; erst `[:k]` wählt sie aus. Der Preis: Die Positionen kommen unsortiert, und bei gleichen Werten an der Auswahlgrenze gibt es keine Regel, welche Position in die Auswahl kommt. Nachsortieren ordnet die gewählten Kandidaten, kann an der Grenze aber nicht mehr korrigieren, welche Kandidaten gewählt wurden. Dazu kommen die Randfälle des Vertrags, `k = 0`, leere Eingabe und `k` größer als die Anzahl, die der Aufruf gesondert behandeln müsste. Deshalb bleibt das Workbook bei `argsort` mit `kind="stable"`. PyTorch bietet mit `torch.topk` eine eigene Top-k-Auswahl; sie liefert Werte und Positionen zusammen und macht ebenfalls keine Zusage zur Reihenfolge bei Gleichständen.

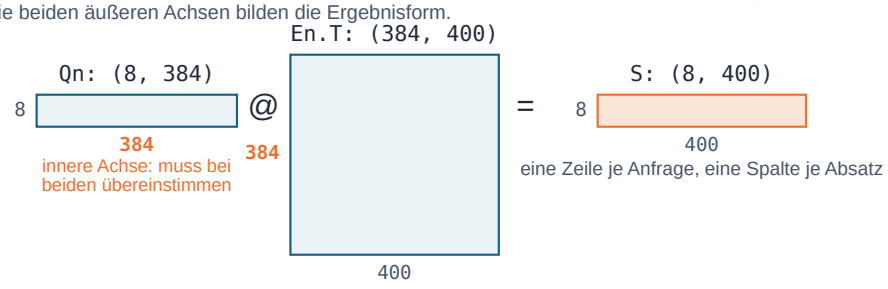
3.6.1 Acht Anfragen auf einmal

> 💭 **Kurz überlegen**
>
> `cosine_scores` aus dem Kapitel 3.5.2 beantwortet eine Anfrage und normalisiert dabei jedes Mal die Anfrage und alle 400 Absätze neu. Für alle acht Anfragen könnte man die Funktion achtmal aufrufen. Oder man normalisiert die Absätze einmal und rechnet `Qn @ En.T` mit allen acht normalisierten Anfragen auf einmal. Wie viel länger dauert die Rechnung für acht Anfragen als für eine? Legen Sie sich auf einen Faktor fest.

Die nächste Zelle baut die Fassung für alle acht Anfragen von Hand aus dem Operator und dem Matrixprodukt, statt `cosine_scores` zu verwenden. Der Grund: `cosine_scores` normalisiert seine Eingaben selbst. Würde man ihm schon normalisierte Daten geben, würden diese ein zweites Mal durch die stabilisierte Formel laufen, und die verändert das Ergebnis bei jedem Durchlauf ein wenig: `[0.0001, 0]` würde erst zu etwa `[0.0995, 0]` und beim zweiten Mal zu `[0.99995, 0]`. Deshalb gilt im ganzen Kurs die Regel: entweder rohe Daten an `cosine_scores`, oder einmal normalisieren und dann nur noch mit `@` rechnen.

Acht Anfragen auf einmal: Qn @ En.T

Beim Matrixprodukt muss die letzte Achse des linken Arrays zur vorletzten des rechten passen.
Die beiden äußeren Achsen bilden die Ergebnisform.



Die innere Achse 384 (die Komponenten der Embeddings) wird aufsummiert und verschwindet; jeder Eintrag von S ist ein Skalarprodukt.
`Eine Anfrage: q (384,) → q[None, :] (1, 384) → (1, 384) @ (384, 400) → (1, 400) → [0] → (400,)`

```
def scores_one(q):
    return cosine_scores(q, E)

def scores_all(Q):
    Qn = row_l2_normalize(Q)                # (8, 384), einmal normalisiert
    En = row_l2_normalize(E)                # (400, 384)
    return Qn @ En.T                        # (8, 384) @ (384, 400) -> (8, 400)

S = scores_all(Q)
print(S.shape, "eine Zeile je Anfrage, eine Spalte je Absatz")
t_one = measure(scores_one, Q[0], repeats=20)
t_all = measure(scores_all, Q, repeats=20)
print(f"eine Anfrage: {t_one * 1000:.3f} ms   acht Anfragen: {t_all * 1000:.3f} ms   Faktor {t_all /
t_one:.1f}")
```

```
(8, 400) eine Zeile je Anfrage, eine Spalte je Absatz
eine Anfrage: 0.157 ms   acht Anfragen: 0.220 ms   Faktor 1.4
```

Vergleichen Sie Ihre Vermutung mit der Messung: Welche Beobachtung stützt sie, welche widerspricht ihr? Der Vorteil gemeinsamer Verarbeitung kann aus zwei Quellen kommen. Vorarbeit, die für alle Anfragen gleich ist, fällt nur einmal an, hier die Normalisierung der Absätze. Und eine Operation auf einem größeren Block ist in kompiliertem Code oft günstiger als mehrere kleine Operationen nacheinander. Wie stark beides wirkt, hängt von Rechner, Bibliothek und Größe der Arrays ab. Das ist Tensor Thinking, wie es das Kapitel 3.3 beschrieben hat: Die Schleife über die Anfragen ist nicht verschwunden, sie steckt in der ersten Achse von `S`.

Schleife gegen NumPy auf dem Messsatz. Zum Schluss des Kapitels die Messung aus der ersten Lerneinheit, jetzt mit Ihrem Operator: Der Messsatz hat 10.000 Zeilen mit je 384 Zufallszahlen, ausdrücklich als `float32`. Die Vorgabe-Fassung mit Schleifen hält denselben Vertrag ein: eine äußere Schleife über die Zeilen, darin zwei aufeinanderfolgende Schleifen über die

Einträge, eine für die Quadratsumme und eine fürs Teilen.

```
    rng = np.random.default_rng(42)
    X = rng.standard_normal((10_000, 384)).astype(np.float32)      # der Messsatz, Form (10000, 384)

    def row_l2_normalize_loop(x, eps=1e-6):
        check_input(x, eps)
        y = np.empty_like(x)                        # gleiche Form und gleicher Typ, Werte noch beliebig
        for i in range(x.shape[0]):
            total = 0.0
            for j in range(x.shape[1]):
                total += x[i, j] * x[i, j]
            length = math.sqrt(total + eps)
            for j in range(x.shape[1]):
                y[i, j] = x[i, j] / length
        return y

    t_loop = measure(row_l2_normalize_loop, X, repeats=1)
    t_numpy = measure(row_l2_normalize, X, repeats=5)
    # erst Korrektheit, dann Zeit
    assert np.allclose(row_l2_normalize_loop(X[:50]), row_l2_normalize(X[:50]), atol=1e-6)
    print(f"Schleife: {t_loop * 1000:8.1f} ms   NumPy: {t_numpy * 1000:6.1f} ms   Faktor {t_loop /
t_numpy:.0f}")
```

```
Schleife:   1642.1 ms   NumPy:    4.8 ms   Faktor 342
```

Die Schleife rechnet in Python mit einzelnen Zahlenobjekten, die sie bei jedem Zugriff `x[i, j]` erst aus dem Block baut. Bei NumPy laufen die einzelnen Operationen in kompiliertem Code; auch dort entstehen mehrere Durchläufe und Zwischenarrays (`x * x`, die Summen, die Wurzeln, der Quotient), aber ohne Interpreter und ohne Zahlenobjekte. Die Mathematik ist in beiden Fassungen dieselbe.

3.6.2 Die ganze Suche am Stück

Alle drei Schritte in einer Zelle, mit Formenzeile. Das ist der Referenzstand, auf dem die Projektaufgabe und die nächsten Lerneinheiten aufbauen. Die Zelle übergibt `cosine_scores` die unnormalisierten Kursdaten `Q[query_no]` und `E`, weil die Funktion selbst normalisiert (siehe die Regel im vorigen Kapitel).

```
    query_no = 3
    q = Q[query_no]                                  # (384,)   der Anfragevektor, roh
    scores = cosine_scores(q, E)                     # (400,)   Normalize und Compare, E bleibt roh
    best = top_k(scores, 3)                          # (3,)     Select: Positionen der drei besten
    print("Anfrage:", queries[query_no]["text"])
    for i in best:
        print(f"  {scores[i]:.3f}  {passages[i]['title']}")
    # Formen: q (384,) -> scores (400,) -> best (3,)
```

```
Anfrage: Wie viel wiegt das Gehirn eines Mannes durchschnittlich?
  0.895  Gehirn
  0.456  Papier
  0.360  Seattle
```

Zum Vergleich die Fassung aus dem Appetizer der ersten Lerneinheit, mit der dortigen NumPy-Funktion `normalize` (nicht der Listenfunktion aus diesem Notebook): `normalize(E) @ normalize(Q[query_no])` und `np.argsort(scores)[::-1][:k]`. Dieselbe Rechnung in zwei Zeilen, aber ohne `eps`, ohne Vertrag und ohne Regel für Gleichstände. Sie wissen jetzt, was in diesen zwei Zeilen steckt.

Vertiefung: %timeit statt measure NICHT PRÜFUNGSRELEVANT

Jupyter bringt eine eigene Kurzform zum Messen mit: `%timeit cosine_scores(Q[0], E)` am Anfang einer Zelle führt den Ausdruck viele Male aus und gibt Mittelwert und Streuung an.

`%timeit` wählt die Anzahl der Wiederholungen selbst und ist für eine schnelle Messung bequem. `measure` aus der ersten Lerneinheit bleibt im Kurs das Werkzeug, weil es in jedem Python-Programm läuft, nicht nur im Notebook, und weil sein Bestwert bei wiederholten Messungen stabiler ist als ein Mittelwert. Der erste Lauf einer Funktion ist oft langsamer als die folgenden, weil Speicher erst angefordert und Daten erst in den Cache geladen werden, den schnellen Zwischenspeicher des Prozessors. `measure` nimmt deshalb das Minimum über mehrere Läufe.

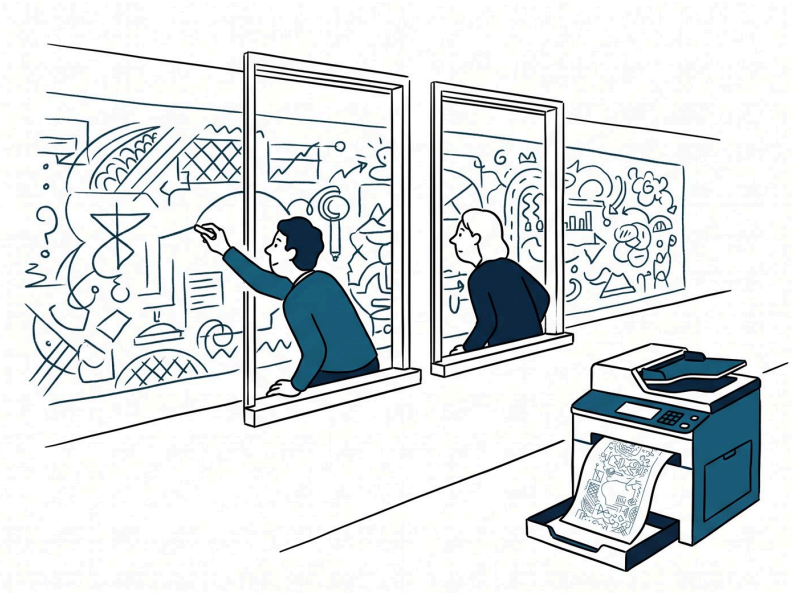
3.7 Sicht oder Kopie

In Grundkurs Teil 2 lieferte ein Slice einer Liste eine neue Liste. Bei Arrays ist das anders: Ein Slice ist eine Sicht (View) auf denselben Datenblock, also eine zweite Beschreibung mit eigener Form und eigenen Abständen, aber ohne eigene Zahlen. Schreibt eine Zelle in die Sicht, ändern sich die Zahlen des Originals. Das spart Speicher und Zeit, weil kein Block kopiert wird, und es ist die Quelle eines typischen Fehlers: Eine Funktion verändert Daten, die der Aufrufer noch braucht. Der Vertrag des Operators verlangt deshalb ausdrücklich, dass die Eingabe unverändert bleibt.

```
a = np.arange(6.0)                  # [0, 1, 2, 3, 4, 5]
b = a[1:4]                          # Sicht auf die Einträge 1 bis 3
b[0] = 99.0                         # schreibt in die Sicht
print(a)                            # das Original hat sich geändert
print(np.shares_memory(a, b))   ①
c = a[1:4].copy()               ②
c[0] = -1.0
print(a, np.shares_memory(a, c))
```

① `np.shares_memory` prüft, ob zwei Arrays mindestens einen Speicherbereich gemeinsam verwenden
② `.copy()` legt einen eigenen Datenblock an

```
[ 0. 99.  2.  3.  4.  5.]
True
[ 0. 99.  2.  3.  4.  5.] False
```



Zwei Fensterrahmen, eine Wandzeichnung: Eine Sicht zeigt denselben Datenblock, jeder neue Strich ist durch beide Rahmen zu sehen. Die Fotokopie ist die Kopie mit eigenem Datenblock, dort ändert sich nichts mit.

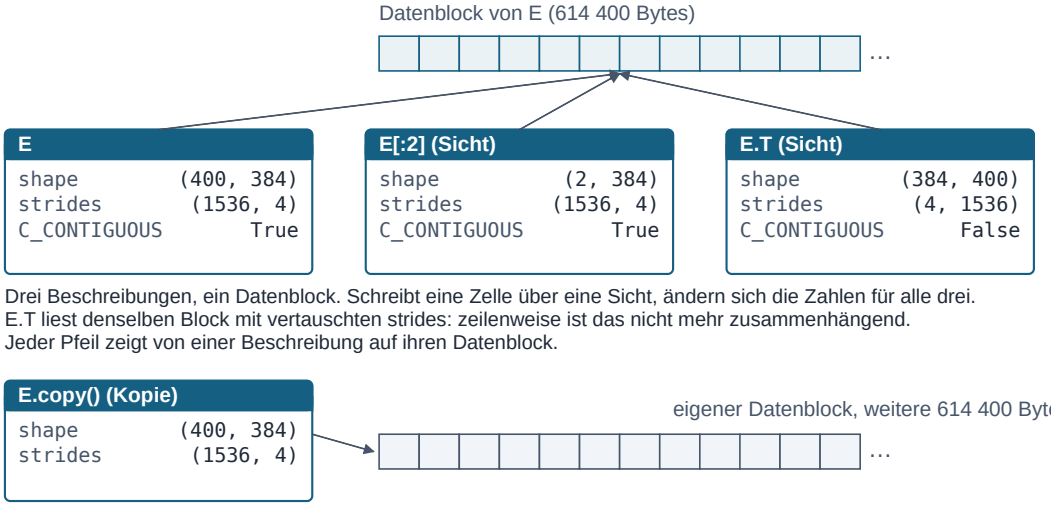
Beim Lesen mit einem Indexarray oder einer Maske entsteht dagegen eine Kopie: `scores[best]` und `scores[mask]` haben eigene Datenblöcke, `scores[:3]` ist eine Sicht. Auch jede Rechnung wie `x / norms` legt ein neues Array an. Das unterscheidet die beiden Schreibweisen, die der Vertrag auseinanderhält: `x = x / norms` bindet den Namen an ein neues Array, das Original bleibt unverändert. `x /= norms` schreibt in das vorhandene Array. Ein Operator, der so rechnet, verletzt den Punkt „Die Eingabe bleibt unverändert", und das fällt erst auf, wenn eine spätere Zelle die Rohdaten braucht.

```
x = np.array([[3.0, 4.0]], dtype=np.float32)
y = x / 5.0                          # neues Array, x bleibt
print(x, np.shares_memory(x, y))
x /= 5.0                             ①
print(x)
print(np.shares_memory(scores, scores[:3]), np.shares_memory(scores, scores[best]))
```

① `/=` rechnet im vorhandenen Array, ohne ein neues anzulegen

```
[[3. 4.]] False
[[0.6 0.8]]
True False
```

Sicht oder Kopie: Wie viele Datenblöcke gibt es?



Drei Beschreibungen, ein Datenblock. Schreibt eine Zelle über eine Sicht, ändern sich die Zahlen für alle drei. E.T liest denselben Block mit vertauschten strides: zeilenweise ist das nicht mehr zusammenhängend. Jeder Pfeil zeigt von einer Beschreibung auf ihren Datenblock.

Eine Kopie entsteht bei .copy(), astype, beim Lesen mit einem Indexarray oder einer Maske und bei jeder Rechnung wie x / norms. Prüfen lässt sich das mit np.shares_memory(a, b).

Zeilenweise zusammenhängend. Auch `E.T` ist eine Sicht: Es verwendet denselben Block, liest ihn aber mit vertauschten Abständen `(4, 1536)` statt `(1536, 4)`, also spaltenweise (Kapitel 3.3.1). Die Zahlen einer Zeile von `E.T` liegen im Speicher deshalb nicht hintereinander, sondern jeweils 1536 Bytes auseinander. NumPy kann damit rechnen; ob das langsamer ist, hängt von der Operation und davon ab, in welcher Richtung sie den Speicher durchläuft. Das Attribut `flags["C_CONTIGUOUS"]` sagt, ob die Zeilen eines Arrays dicht hintereinander im Speicher liegen, so wie die Sprache C ein zweidimensionales Array ablegt. `np.ascontiguousarray` liefert eine zeilenweise zusammenhängende Fassung; liegt das Array schon so vor, gibt es dasselbe Array zurück, es ersetzt `.copy()` also nicht.

Wann `reshape` kopieren muss. Eine Sicht kann nur Formen darstellen, die sich mit einem festen Abstand je Achse auf dem vorhandenen Block beschreiben lassen. `E.reshape(-1)` ist deshalb eine Sicht: Alle 153.600 Zahlen liegen im Block schon zeilenweise hintereinander, der Abstand ist einfach 4 Bytes. `E.T.reshape(-1)` müsste dagegen die Zahlen spaltenweise aneinanderreihen, also erst `E[0, 0]`, dann `E[1, 0]`, dann `E[2, 0]` und nach 400 Schritten mit `E[0, 1]` wieder vorn beginnen. Dafür gibt es keinen festen Abstand, also legt NumPy einen neuen Block mit den Zahlen in dieser Reihenfolge an. Beide Ergebnisse enthalten alle Werte, aber in verschiedener Reihenfolge (zeilenweise gegen spaltenweise); außerdem teilt das erste den Speicher mit `E`, das zweite nicht.

```
print(E.strides, E.T.strides)
print(E.flags["C_CONTIGUOUS"], E.T.flags["C_CONTIGUOUS"], np.shares_memory(E, E.T))
① print(np.shares_memory(E, E.reshape(-1)), np.shares_memory(E, E.T.reshape(-1)))
② ET = np.ascontiguousarray(E.T)
print(ET.flags["C_CONTIGUOUS"], np.shares_memory(E, ET))
print(np.ascontiguousarray(E) is E)
```

① `reshape` liefert eine Sicht, wenn die neue Form zu den Abständen passt, sonst eine Kopie
② `np.ascontiguousarray` kopiert nur, wenn es nötig ist

```
(1536, 4) (4, 1536)
True False True
True False
True False
True
```

Damit lässt sich der letzte Punkt des Vertrags einlösen. Die nächste Vorgabe-Zelle ergänzt `check_input` um die Prüfung auf zeilenweise zusammenhängenden Speicher. Ihre Funktion `row_l2_normalize` ruft `check_input` beim Aufruf auf und bekommt die neue Prüfung damit automatisch. Der Operator ist vollständig.

```
# Vorgabe: check_input mit der letzten Vertragsprüfung — einfach ausführen
def check_input(x, eps):
    if x.ndim != 2:
        raise ValueError(f"Eingabe muss zwei Achsen haben, hat {x.ndim}")
    if x.dtype != np.float32:
        raise ValueError(f"Eingabe muss float32 sein, ist {x.dtype}")
    if x.shape[1] == 0:
        raise ValueError("Eingabe muss mindestens eine Spalte haben (D > 0)")
    eps32 = np.float32(eps)
    if not np.isfinite(eps32) or eps32 <= 0:
        raise ValueError(f"eps muss in float32 positiv und endlich sein, ist {eps}")
    if not x.flags["C_CONTIGUOUS"]:
        raise ValueError("Eingabe muss zeilenweise zusammenhängend sein (np.ascontiguousarray)")

print(row_l2_normalize(np.ascontiguousarray(E.T[:2])).shape, "<- E.T selbst würde abgewiesen")
```

💭 Kurz überlegen

Drei Zellen, dreimal die Frage: Verändert sich `a`? Legen Sie sich für jede fest, bevor Sie die nächste Zelle ausführen.

```
a = np.arange(1.0, 6.0); b = a[2:]; b[0] = 0.0                # 1
a = np.arange(1.0, 6.0); b = a[np.array([2, 3])]; b[0] = 0.0  # 2
a = np.arange(1.0, 6.0); b = a.copy(); b[0] = 0.0             # 3
```

```
# Tragen Sie Ihre Vorhersagen ein (True = a verändert sich), dann ausführen
prediction = {1: None, 2: None, 3: None}
a = np.arange(1.0, 6.0); b = a[2:]; b[0] = 0.0;              changed_1 = a[2] == 0.0
a = np.arange(1.0, 6.0); b = a[np.array([2, 3])]; b[0] = 0.0; changed_2 = a[2] == 0.0
a = np.arange(1.0, 6.0); b = a.copy(); b[0] = 0.0;           changed_3 = a[0] == 0.0
for case, changed in zip((1, 2, 3), (changed_1, changed_2, changed_3)):
    assert prediction[case] == changed, f"Fall {case}: a verändert sich {'' if changed else 'nicht'}"
print("Alle Vorhersagen stimmen ✓")
```

✏️ Aufgabe 3.7 #1 5 min

Tragen Sie für die vier Ausdrücke in der Zelle darunter ein: die Form als Tupel, ob das Ergebnis eine Sicht auf `E` ist, und ob es zeilenweise zusammenhängend ist. Die Prüfzelle vergleicht mit den tatsächlichen Werten.

Wer früher fertig ist: Schreiben Sie in einer neuen Zelle einen Test, der prüft, dass `row_l2_normalize(E.T)` mit einem `ValueError` abbricht. Das Muster mit `try` und `except ValueError` steht in der Prüfzelle zu `row_l2_normalize`.

```
    # Je Ausdruck: Form als Tupel, Sicht auf E (True/False), zeilenweise zusammenhängend (True/False)
    # Beispiel für einen Eintrag:  "E[:5]": ((5, 384), True, True)
    #   E[:5] hat 5 Zeilen und 384 Spalten, ist eine Sicht auf E, und seine Zeilen liegen dicht hintereinander.
    guess = {
        "E[:10]":        (None, None, None),
        "E[:, :10]":     (None, None, None),
        "E.T":           (None, None, None),
        "E[:10].copy()": (None, None, None),
    }
```

```
    # Prüfzelle – einfach ausführen
    actual = {"E[:10]": E[:10], "E[:, :10]": E[:, :10], "E.T": E.T, "E[:10].copy()": E[:10].copy()}
    for name, arr in actual.items():
        shape, view, contiguous = guess[name]
        assert arr.shape == shape, f"{name}: Form {arr.shape}"
        assert np.shares_memory(E, arr) == view, f"{name}: Sicht ist {np.shares_memory(E, arr)}"
        assert arr.flags["C_CONTIGUOUS"] == contiguous, f"{name}: zeilenweise zusammenhängend ist
{arr.flags['C_CONTIGUOUS']}"
    print("Alle Vorhersagen stimmen ✓")
```

Überblick: Sicht oder Kopie

Ausdruck	Ergebnis
Slice `a[1:4]`, `E.T`, `reshape` wenn möglich	Sicht: gleicher Datenblock, Schreiben ändert das Original
`scores[mask]`, `scores[indices]` beim Lesen, `astype` (Voreinstellung `copy=True`), `flatten` (wie `reshape(-1)`, aber immer eine Kopie), jede Rechnung wie `x / norms`	Kopie oder neues Ergebnisarray: eigener Datenblock
`a.copy()`	Kopie, ausdrücklich
`x = x / norms`	neues Array; `x` wird neu gebunden, das ursprüngliche Array bleibt unverändert
`x /= norms`	ändert das vorhandene Array; andere Namen und Sichten darauf sehen die Änderung

Prüfen mit `np.shares_memory(a, b)`. Zeilenweise zusammenhängend: `a.flags["C_CONTIGUOUS"]`; `np.ascontiguousarray(a)` stellt es her und kopiert nur, wenn nötig.

Vertiefung: Was NumPy außerdem mitbringt NICHT PRÜFUNGSRELEVANT

Für einige Muster bringt NumPy fertige Werkzeuge mit, die in dieser Lerneinheit nicht gebraucht werden. In fremdem Code und in späteren Lerneinheiten begegnen sie Ihnen, deshalb zeigt diese Box je ein Beispiel.

Elementweise auswählen. `np.where(mask, a, b)` nimmt an jeder Position den Eintrag aus `a`, wo die Maske `True` ist, sonst den aus `b`:

```
s = np.array([0.1, 0.9, 0.5])
np.where(s >= 0.5, s, 0.0)          # [0.0, 0.9, 0.5]: kleine Werte auf 0 gesetzt
```

Summen über Achsen in Indexschreibweise. `np.einsum` beschreibt mit einer Zeichenkette, welche Achsen multipliziert und welche aufsummiert werden. `"ij,ij->i"` ist das Skalarprodukt je Zeile, `"qd,nd->qn"` das Matrixprodukt `Qn @ En.T`:

```
Qn, En = row_l2_normalize(Q), row_l2_normalize(E)
np.einsum("ij,ij->i", E, E)         # Quadratsummen je Zeile, Form (400,)
np.einsum("qd,nd->qn", Qn, En)      # dasselbe wie Qn @ En.T, Form (8, 400)
```

Mehrere Indexarrays zugleich. Ein Indexarray je Achse wählt beliebige Einträge einer Matrix aus; das Puzzle `diag` im Kapitel 3.8 zeigt den einfachsten Fall:

```
rows = np.array([0, 2])
cols = np.array([1, 3])
M[rows, cols]                       # [M[0, 1], M[2, 3]] = [2, 12]
```

3.8 Zum Weiterüben

Die folgenden Aufgaben sind Tensor Puzzles nach Sasha Rush (Cornell Tech), auf NumPy übertragen. Das Original ist eine Sammlung von 21 Rätseln für PyTorch, frei verfügbar unter github.com/srush/Tensor-Puzzles (MIT-Lizenz, siehe Quellenblock). Jede Aufgabe baut ein bekanntes Werkzeug von NumPy mit Broadcasting, Masken und Indexarrays nach, ohne Schleife und in einer Zeile. Die Einzeiler-Regel ist eine Übungsbedingung wie im Original, keine Empfehlung für möglichst kompakten Code. Lesbarer Code mit mehreren Zeilen ist im Kurs immer erlaubt.

Erlaubte Werkzeuge, je mit einem Beispiel:

- `np.arange(n)` zählt von 0 bis n−1
- Vergleiche liefern Masken: `np.arange(4) < 2` ist `[True, True, False, False]`
- `[:, None]` und `[None, :]` fügen Achsen ein: `a[:, None] * b[None, :]` ist das Produkt jedes Eintrags von `a` mit jedem von `b`
- Rechenzeichen und `@`, `sum` mit `axis=`
- zwei Indexarrays gleichzeitig: `A[np.array([0, 1]), np.array([1, 0])]` liefert `A[0, 1]` und `A[1, 0]`

Verboten ist jeweils das Werkzeug, das nachgebaut wird (`np.ones`, `np.outer`, `np.eye`, `np.diag`). Die Lösungen erscheinen mit den Lösungen des Termins.

✏️ Aufgabe 3.8 #1 Weiterüben

Puzzle 1: ones. Schreiben Sie `ones(n)`, das ein Array aus `n` Einsen liefert, ohne `np.ones` und ohne `np.full`.

```
def ones(n):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
assert np.array_equal(ones(4), [1, 1, 1, 1]) and ones(0).shape == (0,)
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 3.8 #2 Weiterüben

Puzzle 2: outer. Schreiben Sie `outer(a, b)`, das für zwei eindimensionale Arrays die Matrix aller Produkte `a[i] * b[j]` liefert, Form `(len(a), len(b))`, ohne `np.outer` und ohne `@`.

```
def outer(a, b):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
assert np.array_equal(outer(np.array([1, 2]), np.array([10, 20, 30])), [[10, 20, 30], [20, 40, 60]])
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 3.8 #3 Weiterüben

Puzzle 3: eye. Schreiben Sie `eye(n)`, das die Einheitsmatrix der Form `(n, n)` liefert (Einsen auf der Diagonalen, sonst Nullen), ohne `np.eye` und ohne `np.identity`. Eine Maske lässt sich mit `* 1` in Zahlen umwandeln.

```
def eye(n):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
assert np.array_equal(eye(3), [[1, 0, 0], [0, 1, 0], [0, 0, 1]])
print("Alle Prüfungen bestanden ✓")
```

✏️ Aufgabe 3.8 #4 Weiterüben

Puzzle 4: diag. Schreiben Sie `diag(A)`, das die Diagonale einer quadratischen Matrix als eindimensionales Array liefert, ohne `np.diag` und ohne `np.diagonal`.

```
def diag(A):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
assert np.array_equal(diag(np.array([[1, 2], [3, 4]])), [1, 4])
print("Alle Prüfungen bestanden ✓")
```

➡️ Nächste Schritte

- Beenden Sie die Aufgaben, die im Termin offen geblieben sind. Zur Schlusskontrolle starten Sie die Laufzeit neu und führen die Zellen bis zum Ende des Kapitels 3.7 aus: Alle Prüfzellen bis dorthin sollten mit Ihren eigenen Lösungen durchlaufen. Die Prüfzellen im Kapitel „Zum Weiterüben" zählen nur, wenn Sie die Puzzles bearbeitet haben.
- In der Woche ohne Termin folgen Teil 3 des Python-Grundkurses (Dateien, Fehlerbehandlung, Funktionen, Module und Klassen) und die Projektaufgabe: ein kleines Suchwerkzeug von der Datei bis zur geprüften Trefferliste, aufgebaut auf der Suche aus diesem Notebook. Beides kommt mit der nächsten E-Mail.
- Die Referenzkarte NumPy fasst NumPy auf zwei Seiten zusammen.

→ **Ausblick:** In der nächsten Lerneinheit kommen pandas und Matplotlib: Daten einlesen, bereinigen, zusammenfassen und darstellen.

Quellen und Referenzen

- J. VanderPlas: *Python Data Science Handbook*, 2. Auflage, Kapitel 4 bis 11 (NumPy: Datentypen, Arrays, Ufuncs, Aggregationen, Broadcasting, Masken, Fancy Indexing, Sortieren). Online frei verfügbar.
- W. McKinney: *Python for Data Analysis*, 3. Auflage, Kapitel 4 und Anhang A (ndarray-Interna, strides, Broadcasting, partielles Sortieren, zusammenhängender Speicher).
- M. Gorelick, I. Ozsvald: *High Performance Python*, 3. Auflage, Kapitel 6 (Zwischenresultate, In-Place-Operationen).
- NumPy-Dokumentation: Broadcasting, Copies and views, Indexing, `ndarray.flags`, `numpy.argsort`.
- C. R. Harris et al.: *Array programming with NumPy*, Nature 585 (2020), 357–362.
- S. Rush: Tensor Puzzles (MIT-Lizenz). Die Aufgaben im Kapitel „Zum Weiterüben" folgen dieser Idee und sind auf NumPy übertragen.
- Datengrundlage: GermanQuAD (deepset), Lizenz CC BY 4.0. Die Absatztexte stammen aus der deutschen Wikipedia (CC BY-SA). Attribution auch in `data/appetizer_passages.json`.

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT, Karikaturen: Google Gemini). Alle Inhalte sind von Prof. Dr. Stephan Kurpjuweit konzipiert, geprüft und verantwortet.